ДОКУМЕНТАЦИЯ ПО ЭТАПАМ

учебной практики

«Разработка модуля учёта заявок на ремонт транспорта
с базой данных и статусами, с расчётом среднего времени ремонта»

Специальность 09.02.07 «Информационные системы и программирование»

Выполнил(а): _____

Группа: _____

Руководитель: _____

2026

СОДЕРЖАНИЕ

Этап 01. Вводное занятие и инструктаж по технике безопасности 5

Этап 02. Анализ предметной области 7

Этап 03. Составление технического задания по ГОСТ 19.201-78 9

Этап 04. Структурная схема предприятия 11

Этап 05. UML-диаграмма вариантов использования 13

Этап 06. UML-диаграмма классов 15

Этап 07. ER-диаграмма базы данных 17

Этап 08. Нормализация схемы базы данных до третьей нормальной формы 19

Этап 09. Правила ссылочной целостности 21

Этап 10. Создание проекта WinForms и структуры решения 23

Этап 11. Настройка Git-репозитория 25

Этап 12. Создание базы данных SQLite и миграции 27

Этап 13. Модель User и таблица Users 29

Этап 14. Реализация формы входа 31

Этап 15. Разграничение ролей пользователей 33

Этап 16. Справочник транспортных средств и неисправностей 35

Этап 17. Разработка формы справочника транспортных средств 37

Этап 18. Поиск и валидация в справочнике транспортных средств 39

Этап 19. Модель заявки на ремонт и история статусов 41

Этап 20. Форма регистрации заявки на ремонт 43

Этап 21. Интеграция формы заявки со справочниками 45

Этап 22. Слой доступа к данным 47

Этап 23. Запросы с фильтрацией и сортировкой 49

Этап 24. Обеспечение ссылочной целостности при работе с заявками 51
Этап 25. Фильтрация заявок по диапазону дат 53
Этап 26. Фильтрация заявок по клиенту..... 55
Этап 27. Сортировка и постраничный вывод заявок..... 57
Этап 28. Постановка задачи статистики времени ремонта..... 59
Этап 29. Реализация расчёта статистики в коде 61
Этап 30. Интеграция расчёта статистики в форму отчёта 63
Этап 31. Вывод статистики в экспортируемых отчётах 65
Этап 32. Подключение журналирования Serilog 67
Этап 33. Конфигурация приложения 69
Этап 34. Интеграция с внешним API курсов валют 71
Этап 35. Модульные тесты авторизации и валидации..... 73
Этап 36. Интеграционные тесты взаимодействия заявки и справочников
..... 75
Этап 37. Разработка тест-плана 77
Этап 38. Отчёт по заявкам за период 79
Этап 39. Экспорт отчёта в Excel..... 81
Этап 40. Экспорт отчёта в PDF..... 83
Этап 41. Построение графиков по данным базы 85
Этап 42. Анализ статистики и оформление графиков в отчёте..... 87
Этап 43. Руководство пользователя 89
Этап 44. Описание программы по ГОСТ 19.402-78 91
Этап 45. Оформление листингов ключевых фрагментов кода..... 93
Этап 46. Сборка релиза и публикация приложения 95
Этап 47. Инструкция по развёртыванию 97

Этап 48. Инструкция по технике безопасности 99

Этап 49. Проверка структуры проекта и исправление ошибок..... 101

Этап 50. Рефакторинг кода..... 103

Этап 51. Исправление замечаний и подготовка документации 105

Этап 52. Подготовка презентации к защите..... 107

Этап 53. Пробное выступление и подготовка ответов на вопросы 109

Этап 54. Защита отчёта и демонстрация работы системы..... 111

Этап 01. Вводное занятие и инструктаж по технике безопасности

1. Цель и задачи этапа

Цель этапа — Ознакомиться с целями и задачами учебной практики, порядком выполнения работ и требованиями охраны труда при работе с персональным компьютером.

Для достижения цели решались следующие задачи:

1. изучить программу практики и перечень этапов;
2. рассмотреть структуру итогового отчёта и требования к оформлению документов;
3. пройти вводный инструктаж по технике безопасности;
4. подготовить рабочее место и проверить условия труда.

2. Ход выполнения работ

Учебная практика проходила на базе учебной мастерской, оснащённой персональными компьютерами под управлением Windows 11. Тема практики — разработка модуля учёта заявок на ремонт транспорта с базой данных и статусами, а также расчёт среднего времени ремонта. На вводном занятии были разъяснены цели практики: закрепление навыков проектирования информационных систем, разработки приложения на языке C# с использованием Windows Forms, работы с базой данных SQLite и статистической обработки накопленных данных.

Была рассмотрена структура отчётных документов. Итоговый отчёт включает введение, семь разделов, заключение, список использованных источников и приложения А–Д. Отдельно по каждому этапу практики оформляется краткий документ, в котором указываются цель, задачи, ход выполнения работ и полученный результат. Такой порядок позволяет проверять ход практики последовательно, а не только по завершении всех работ.

При проведении инструктажа по технике безопасности были изучены правила включения и выключения компьютера, требования к освещению рабочего места и расстоянию до монитора. Сообщено о необходимости делать перерывы по 10–15 минут каждый час работы, о запрете работы при повреждённых кабелях и открытых корпусах техники, а также о порядке действий при возгорании и правилах эвакуации.

Рабочее место было подготовлено: проверены целостность сетевого кабеля и кабелей периферийных устройств, устойчивость стола и стула, наличие заземления в розетке. Замечаний по организации рабочего места не выявлено, рабочее место допущено к работе.

Таблица 1 — Календарный план практики

Этапы	Содержание работ
01–09	Анализ предметной области и проектирование системы
10–18	Создание проекта, базы данных, авторизация и справочники
19–27	Разработка модуля заявок и работа с данными
28–31	Математическая обработка времени ремонта
32–42	Интеграции, тестирование и отчётность
43–48	Документирование и развёртывание
49–54	Завершение практики, презентация и защита

3. Полученный результат

Составлен календарный план практики, изучены требования к отчётным документам, рабочее место подготовлено и проверено. Замечаний по условиям труда нет.

4. Вывод

Вводный этап создаёт организационную основу дальнейшей работы: определены сроки, состав документов и безопасные условия выполнения практики.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 02. Анализ предметной области

1. Цель и задачи этапа

Цель этапа — Описать деятельность станции технического обслуживания, состав пользователей и основные процессы, обосновать необходимость разработки информационной системы.

Для достижения цели решались следующие задачи:

1. изучить порядок приёма и выполнения заявок на ремонт;
2. выделить роли пользователей и их права;
3. описать существующий и предлагаемый процессы обработки заявки;
4. определить границы автоматизации.

2. Ход выполнения работ

Предметной областью является деятельность станции технического обслуживания «АвтоМастер», которая выполняет диагностику, слесарный и кузовной ремонт, шиномонтаж и плановое обслуживание автомобилей. Клиент обращается на приёмку, сообщает сведения об автомобиле и неисправности, после чего заявка передаётся механику. По завершении работ оформляются стоимость и фактическое время ремонта, автомобиль выдаётся владельцу.

При бумажной организации учёта заявки записываются в журнал приёмки. Такой порядок затрудняет поиск нужной записи, не позволяет быстро определить загруженность механиков и не даёт возможности получить достоверную статистику времени ремонта. Кроме того, при утере журнала или неразборчивой записи теряется история обслуживания конкретного автомобиля.

В системе выделены три роли. Клиент видит только свои автомобили и свои заявки, может создать заявку и отслеживать её состояние. Менеджер принимает заявки, назначает механика, изменяет статусы и формирует отчёты. Администратор дополнительно управляет справочниками и

учётными записями. Неавторизованный посетитель может только войти в систему или зарегистрироваться.

Предлагаемый процесс включает регистрацию клиента, добавление автомобиля в справочник, оформление заявки с выбором неисправности, назначение исполнителя, последовательную смену статусов и закрытие заявки с указанием стоимости. Границы автоматизации охватывают учёт заявок, справочники транспорта и неисправностей, расчёт статистики и формирование отчётов. Бухгалтерский учёт, складской учёт запчастей и расчёт заработной платы в разрабатываемую систему не входят.

3. Полученный результат

Описаны деятельность станции, роли пользователей и жизненный цикл заявки. Определены границы системы и обоснована необходимость её разработки.

4. Вывод

Анализ предметной области показал, что основная проблема связана с отсутствием единого учёта заявок и статистики, поэтому автоматизация приёмки и выполнения ремонтов является обоснованной.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 03. Составление технического задания по ГОСТ 19.201-78

1. Цель и задачи этапа

Цель этапа — Разработать техническое задание на информационную систему в соответствии со структурой ГОСТ 19.201-78.

Для достижения цели решались следующие задачи:

1. определить назначение и цели создания системы;
2. сформулировать функциональные требования и требования к надёжности;
3. описать стадии и этапы разработки;
4. установить порядок контроля и приёмки.

2. Ход выполнения работ

Техническое задание оформлено по ГОСТ 19.201-78 и включает общие сведения, назначение и цели создания системы, характеристику объекта автоматизации, требования к системе, состав и содержание работ, порядок контроля и приёмки, а также требования к документированию. Полный текст задания приведён в приложении А итогового отчёта.

В разделе «Требования к системе» выделены функциональные требования: авторизация и регистрация пользователей, разграничение прав по ролям, ведение справочников транспортных средств и неисправностей, оформление заявок, смена статусов, назначение механика, расчёт среднего времени ремонта и формирование отчёта по простоям с выгрузкой в Excel и PDF. Отдельно указаны требования к валидации вводимых данных и к сохранению истории изменения статусов.

К системе предъявлены требования по надёжности и безопасности: сохранение данных при аварийном завершении работы, резервное копирование файла базы данных, хранение паролей только в виде хеша, запрет удаления справочных записей, на которые ссылаются заявки.

Требования к эргономике предусматривают единообразное оформление форм, читаемые размеры шрифтов и подтверждение опасных действий.

В техническом задании определены ограничения: приложение является настольным, работает под управлением Windows 10 или 11, использует локальную базу данных SQLite и не требует постоянного подключения к сети. Стадии разработки: проектирование, создание базы данных, разработка модулей, тестирование, документирование и развёртывание.

Таблица 1 — Основные функциональные требования

Функция	Требование
Авторизация	Вход по электронной почте и паролю, проверка хеша
Роли	Клиент, менеджер, администратор; ограничение видимости данных
Справочники	Добавление, изменение, удаление и поиск транспортных средств и неисправностей
Заявки	Создание, назначение механика, смена статусов, закрытие
Статистика	Среднее, медиана, дисперсия времени ремонта
Отчётность	Отчёт за период, экспорт в Excel и PDF

3. Полученный результат

Техническое задание согласовано и оформлено как отдельный документ. Требования к системе используются при дальнейшем проектировании и проверке результата.

4. Вывод

Техническое задание упорядочило требования к системе и позволило перейти к построению схем и диаграмм без дополнительных уточнений.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 04. Структурная схема предприятия

1. Цель и задачи этапа

Цель этапа — Построить структурную схему станции технического обслуживания и показать информационные связи между подразделениями.

Для достижения цели решались следующие задачи:

1. выделить подразделения и должности;
2. определить подчинённость и информационные потоки;
3. построить схему в редакторе draw.io;
4. подготовить описание схемы.

2. Ход выполнения работ

Структурная схема отражает организационное устройство станции технического обслуживания. Во главе находится директор, которому подчиняются служба приёмки, производственный участок, склад запчастей и бухгалтерия. Служба приёмки взаимодействует с клиентами, производственный участок объединяет механиков разных специализаций, склад обеспечивает ремонт запасными частями.

Основной информационный поток начинается с обращения клиента. Менеджер приёмки регистрирует автомобиль и заявку, определяет предварительную стоимость и передаёт заявку на производственный участок. Механик после выполнения работ указывает фактическое время и стоимость, после чего менеджер закрывает заявку и оформляет документы для клиента.

Для построения схемы использован редактор draw.io. Схема сохранена в исходном формате .drawio, что позволяет при необходимости изменить состав подразделений и связи между ними. Для отчёта и презентации схема экспортирована в изображение формата PNG.

На схеме сплошными линиями показана административная подчинённость, пунктирными — передача информации о заявках. Отдельно

отмечено, что склад и бухгалтерия не автоматизируются в рамках разрабатываемого модуля, однако получают сведения о закрытых заявках.

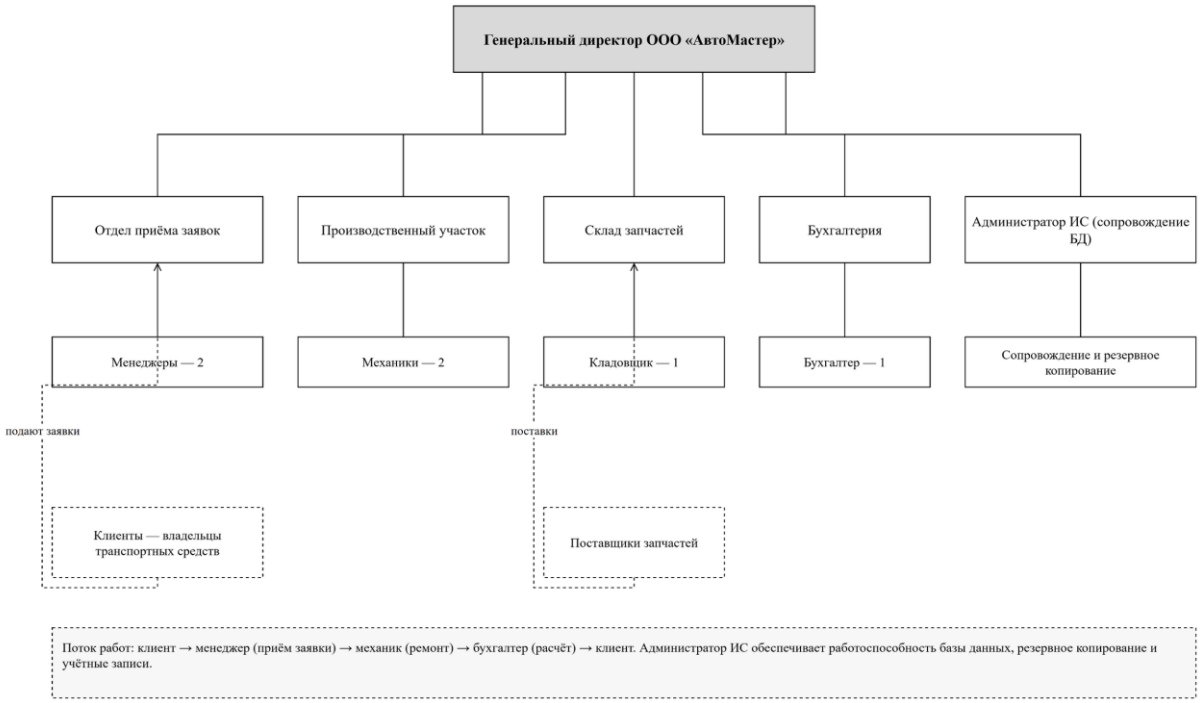


Рисунок 1 — Структурная схема станции технического обслуживания

3. Полученный результат

Структурная схема предприятия построена и сохранена в редактируемом виде. Определены подразделения, участвующие в обработке заявки.

4. Вывод

Схема даёт наглядное представление о месте разрабатываемой системы в деятельности предприятия и о направлении информационных потоков.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 05. UML-диаграмма вариантов использования

1. Цель и задачи этапа

Цель этапа — Разработать диаграмму вариантов использования, отражающую функции системы для каждой категории пользователей.

Для достижения цели решались следующие задачи:

1. определить акторов системы;
2. выделить варианты использования и связи между ними;
3. построить диаграмму в нотации UML;
4. проверить полноту функций по техническому заданию.

2. Ход выполнения работ

Акторами системы являются клиент, менеджер и администратор. Клиент может зарегистрироваться, войти в систему, добавить свой автомобиль, оформить заявку на ремонт, просмотреть состояние своих заявок и сформировать отчёт по своим ремонтам. Менеджер дополнительно обрабатывает поступившие заявки, назначает механика, изменяет статусы и формирует отчёты за период.

Администратор обладает всеми правами менеджера и дополнительно ведёт справочники транспортных средств и неисправностей, управляет учётными записями и блокирует пользователей. Общими для всех авторизованных пользователей являются варианты «Войти в систему» и «Просмотреть отчёт», при этом состав отчёта зависит от роли.

Диаграмма построена в редакторе draw.io. Для вариантов использования применены овалы, для акторов — фигуры человечков, связи обозначены линиями ассоциации. Связи include использованы для входа в систему и проверки прав, поскольку эти действия выполняются при обращении к любой защищённой функции.

Полнота диаграммы проверена по перечню функциональных требований технического задания. Каждое требование соотнесено хотя бы с

одним вариантом использования, лишних функций на диаграмме нет. Исходный файл сохранён в формате .drawio, изображение включено в приложение Б отчёта.

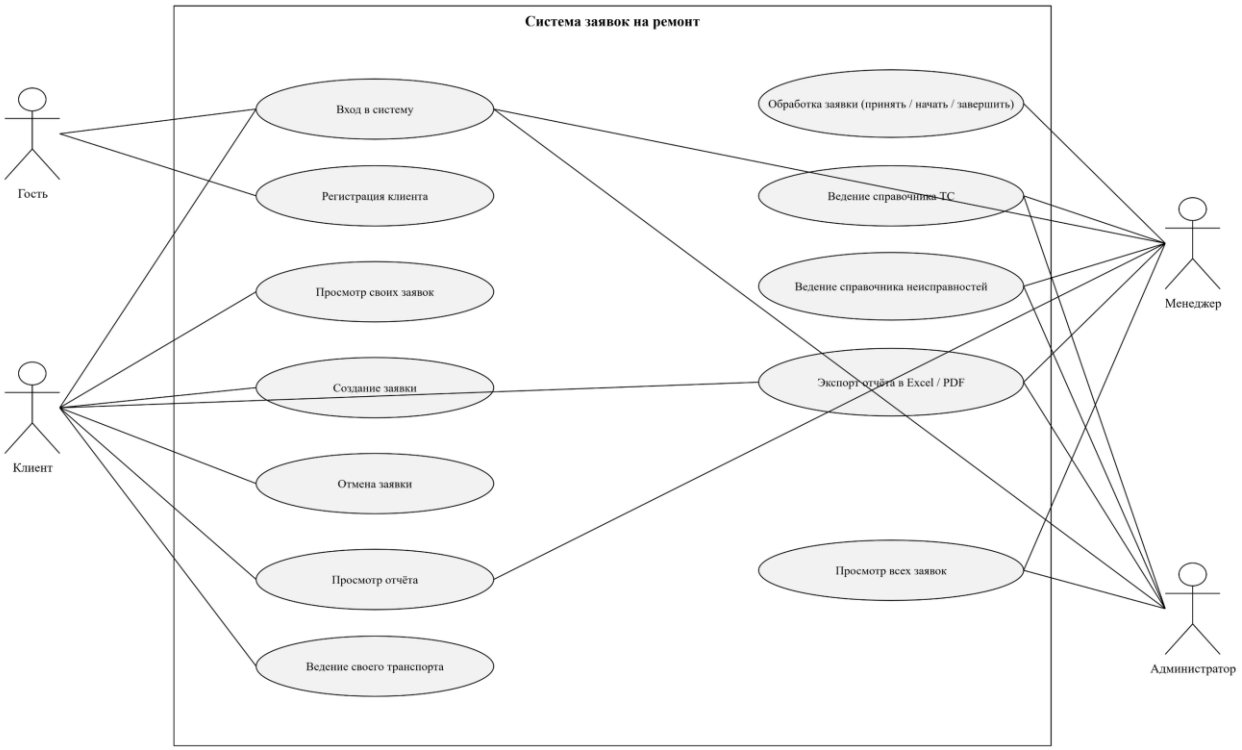


Рисунок 1 — Диаграмма вариантов использования

3. Полученный результат

Диаграмма вариантов использования построена для трёх ролей и содержит основные функции системы. Состав функций соответствует техническому заданию.

4. Вывод

Диаграмма позволила согласовать границы системы и распределение прав между пользователями до начала программирования.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 06. UML-диаграмма классов

1. Цель и задачи этапа

Цель этапа — Построить диаграмму классов, отражающую состав сущностей, их атрибуты, методы и связи.

Для достижения цели решались следующие задачи:

1. выделить классы предметной области;
2. определить атрибуты и методы классов;
3. установить виды связей и кратности;
4. оформить диаграмму в нотации UML.

2. Ход выполнения работ

Основными классами предметной области являются User, Vehicle, FaultType, RepairRequest и RequestStatusHistory. Класс User описывает учётную запись и содержит идентификатор, ФИО, электронную почту, хеш пароля, роль, телефон и признак блокировки. Класс Vehicle хранит марку, модель, государственный номер, VIN, год выпуска и пробег автомобиля.

Класс FaultType относится к справочнику неисправностей и содержит наименование, категорию, описание, норматив трудоёмкости и базовую стоимость работ. Центральный класс RepairRequest объединяет ссылки на автомобиль, неисправность, клиента и механика, а также описание, приоритет, статус, даты жизненного цикла и стоимости работ и запчастей.

Класс RequestStatusHistory предназначен для хранения истории изменения статусов. Он содержит ссылку на заявку, новое значение статуса, дату изменения, автора изменения и комментарий. Служебные классы AuthService, RequestWorkflow и RepairStatisticsService вынесены отдельно: они реализуют проверку учётных данных, правила смены статусов и расчёт статистики.

На диаграмме показаны связи «один ко многим»: пользователь владеет несколькими автомобилями, автомобиль может участвовать в нескольких

заявках, одна неисправность может быть указана в разных заявках. Заявка имеет много записей истории, а каждая запись относится только к одной заявке. Кратности и направления связей соответствуют схеме базы данных.

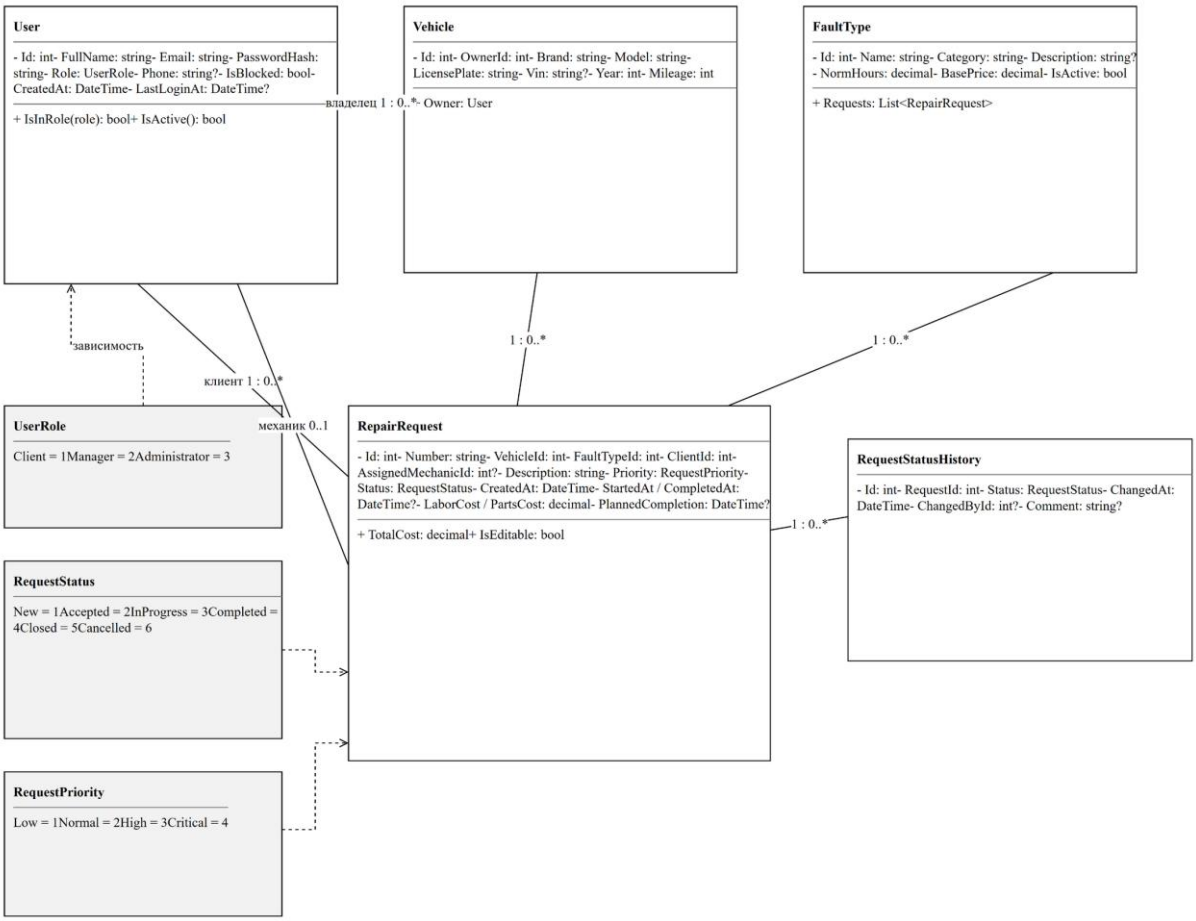


Рисунок 1 — Диаграмма классов

3. Полученный результат

Диаграмма классов построена и согласована со структурой базы данных. Определён состав моделей и сервисов будущего приложения.

4. Вывод

Диаграмма классов стала основой для создания моделей Entity Framework Core и разработки форм интерфейса.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 07. ER-диаграмма базы данных

1. Цель и задачи этапа

Цель этапа — Спроектировать логическую структуру базы данных и представить её в виде ER-диаграммы.

Для достижения цели решались следующие задачи:

1. определить сущности и их атрибуты;
2. выделить первичные и внешние ключи;
3. установить кратности связей;
4. построить ER-диаграмму в редакторе draw.io.

2. Ход выполнения работ

База данных включает пять таблиц: Users, Vehicles, FaultTypes, RepairRequests и RequestStatusHistory. Первичные ключи таблиц имеют целочисленный тип и автоматически увеличиваются. Для электронной почты пользователя, государственного номера автомобиля и номера заявки заданы уникальные индексы.

Связь Users — Vehicles имеет кратность один ко многим: один клиент может владеть несколькими автомобилями, но каждый автомобиль принадлежит одному владельцу. Связь Vehicles — RepairRequests также один ко многим: автомобиль может ремонтироваться несколько раз, но каждая заявка относится к одному автомобилю.

Заявка связана с клиентом и механиком через внешние ключи ClientId и AssignedMechanicId. Механик может быть не назначен, поэтому поле AssignedMechanicId допускает значение NULL. Связь с неисправностью обязательна: каждая заявка оформляется по конкретному виду неисправности из справочника.

История статусов связана с заявкой отношением один ко многим и удаляется каскадно вместе с ней. Для ускорения поиска созданы индексы по дате создания заявки, статусу и составной индекс по заявке и дате изменения

истории. ER-диаграмма сохранена в формате .drawio и включена в приложение Б.

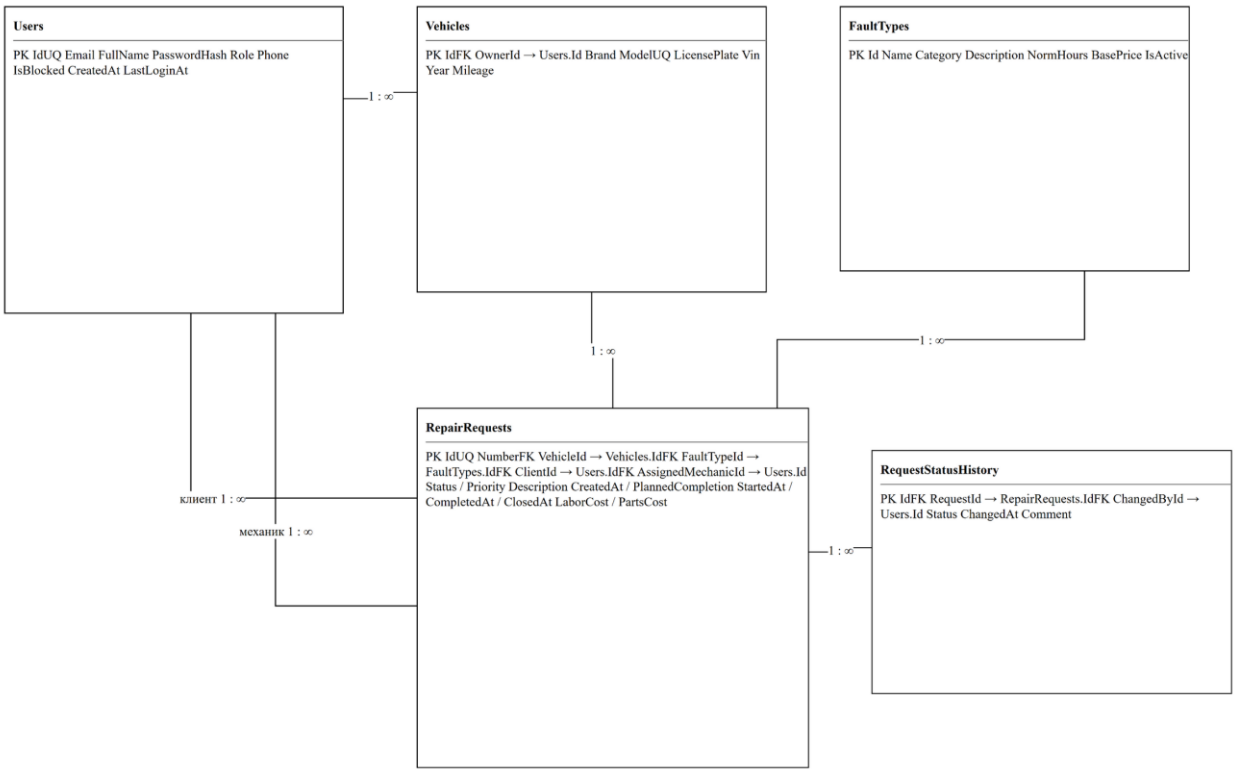


Рисунок 1 — ER-диаграмма базы данных

3. Полученный результат

ER-диаграмма построена, определены пять таблиц, их ключи и связи. Структура базы данных соответствует диаграмме классов.

4. Вывод

ER-диаграмма позволила перейти к созданию контекста Entity Framework Core и миграции базы данных.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 08. Нормализация схемы базы данных до третьей нормальной формы

1. Цель и задачи этапа

Цель этапа — Проверить схему базы данных на соответствие третьей нормальной форме и устранить избыточность данных.

Для достижения цели решались следующие задачи:

1. проверить атомарность атрибутов;
2. проверить зависимость неключевых атрибутов от первичного ключа;
3. выявить транзитивные зависимости;
4. при необходимости разделить таблицы.

2. Ход выполнения работ

Требование первой нормальной формы означает, что все атрибуты атомарны. В таблицах отсутствуют составные поля и повторяющиеся группы: адрес не разделяется на отдельные столбцы, а сведения о владельце автомобиля хранятся в таблице Users, а не в одной строке с автомобилем.

Вторая нормальная форма требует, чтобы неключевые атрибуты зависели от полного первичного ключа. Все таблицы имеют простой суррогатный первичный ключ Id, поэтому частичных зависимостей возникнуть не может. Например, марка и модель автомобиля зависят только от идентификатора автомобиля.

Третья нормальная форма запрещает транзитивные зависимости. Наименование и нормативная трудоёмкость неисправности хранятся в таблице FaultTypes, а не в каждой заявке. ФИО и телефон клиента находятся в Users, а заявка содержит только внешний ключ ClientId. Благодаря этому при изменении справочных данных не требуется обновлять множество строк.

Расчётные величины, такие как общая стоимость заявки и длительность ремонта, в базе данных не хранятся. Они вычисляются в приложении по сохранённым датам и стоимостям, что исключает противоречие между

исходными и расчётными данными. Схема приведена к третьей нормальной форме, избыточность устранена.

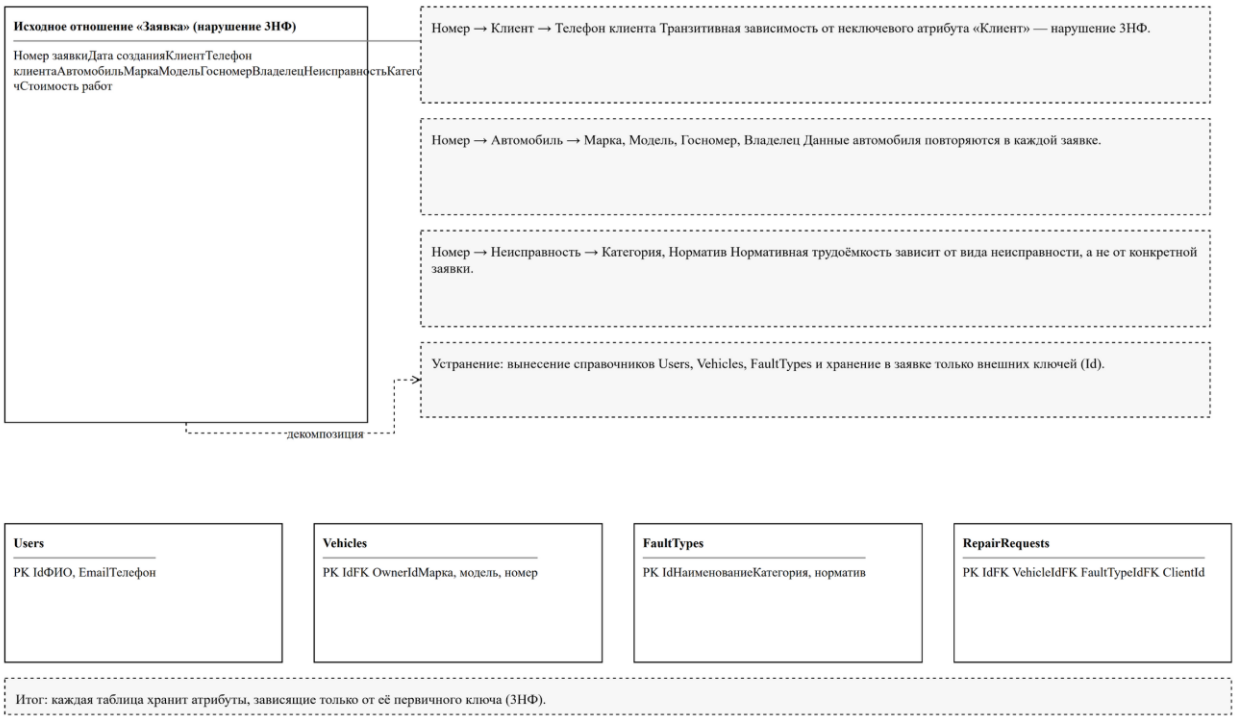


Рисунок 1 — Схема базы данных в третьей нормальной форме

3. Полученный результат

Схема базы данных проверена по трём нормальным формам, транзитивные зависимости отсутствуют, справочные данные вынесены в отдельные таблицы.

4. Вывод

Нормализация обеспечила непротиворечивость данных и упростила сопровождение справочников.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 09. Правила ссылочной целостности

1. Цель и задачи этапа

Цель этапа — Описать правила ссылочной целостности и ограничения, обеспечивающие корректность данных при удалении и изменении записей.

Для достижения цели решались следующие задачи:

1. определить поведение внешних ключей при удалении;
2. назначить ограничения на значения полей;
3. описать проверки на уровне приложения;
4. проверить правила при работе с формами.

2. Ход выполнения работ

Ссылочная целостность обеспечивается внешними ключами. Для справочных данных применено ограничение Restrict: нельзя удалить клиента, у которого есть автомобили или заявки, нельзя удалить автомобиль, по которому уже оформлялись заявки, и нельзя удалить неисправность, указанную хотя бы в одной заявке. Такое поведение защищает историю ремонтов от случайного удаления.

История изменения статусов является частью заявки, поэтому для связи RequestStatusHistory — RepairRequests выбрано каскадное удаление. При удалении заявки связанные записи истории удаляются автоматически. Для механика использовано ограничение Restrict: удаление пользователя, на которого ссылаются заявки, запрещено.

На уровне базы данных заданы уникальные индексы для электронной почты, государственного номера и номера заявки. Проверки диапазонов выполняются средствами Entity Framework Core и атрибутами валидации моделей: год выпуска ограничен диапазоном 1950–2100, пробег и стоимости не могут быть отрицательными, норматив трудоёмкости — от 0,1 до 200 часов.

Дополнительно проверки выполняются в формах до сохранения записи. Если пользователь пытается удалить справочную запись, связанную с заявками, приложение выводит объяснение и предлагает не удалять запись, а сделать неисправность неактивной. Это позволяет сохранить историю и убрать устаревшую позицию из выбора при оформлении новых заявок.

Таблица 1 — Правила удаления связанных записей

Связь	Правило	Обоснование
Users — Vehicles	Restrict	Нельзя удалить владельца с автомобилями
Vehicles — RepairRequests	Restrict	Сохраняется история ремонтов
FaultTypes — RepairRequests	Restrict	Неисправность используется в заявках
RepairRequests — History	Cascade	История является частью заявки
Users — AssignedRequests	Restrict	Сохраняется авторство работ

3. Полученный результат

Правила ссылочной целостности реализованы в модели данных и проверены при работе с формами справочников и заявок.

4. Вывод

Ограничения исключают появление «висячих» ссылок и потерю истории ремонтов при удалении справочных записей.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 10. Создание проекта WinForms и структуры решения

1. Цель и задачи этапа

Цель этапа — Создать решение и настроить структуру проектов для разделения моделей, интерфейса и тестов.

Для достижения цели решались следующие задачи:

1. создать решение и проекты библиотеки, приложения и тестов;
2. настроить целевую платформу и зависимости;
3. разложить классы по каталогам;
4. проверить сборку и запуск пустой формы.

2. Ход выполнения работ

Проект создан на платформе .NET 10. Решение RepairRequests.slnx включает три проекта. RepairRequests.Core содержит модели данных, контекст базы данных, репозитории и сервисы и не зависит от интерфейса. RepairRequests.WinForms содержит формы и вспомогательные классы оформления. RepairRequests.Tests предназначен для модульных и интеграционных тестов.

В проекте Core подключены пакеты Microsoft.EntityFrameworkCore.Sqlite и Microsoft.EntityFrameworkCore.Design, в приложении — Serilog и Serilog.Sinks.File для журналирования, ClosedXML для выгрузки в Excel и QuestPDF для формирования PDF-отчёта. В тестовом проекте использованы xUnit и Microsoft.EntityFrameworkCore.Sqlite для проверки запросов на реальной базе данных.

Каталоги проекта Core разделены по назначению: Models — сущности и перечисления, Data — контекст и заполнение базы, Repositories — доступ к данным, Services — бизнес-логика, экспорт и работа с внешним сервисом курсов валют. В проекте WinForms выделены каталоги Forms и Infrastructure. Такое разделение упрощает поиск нужного класса и позволяет тестировать логику отдельно от форм.

После создания проектов выполнена проверка сборки командой dotnet build. Ошибок и предупреждений нет. Пустая форма запускается, заголовок приложения и размеры окна заданы в соответствии с выбранным оформлением.

Таблица 1 — Состав решения

Проект	Назначение	Платформа
RepairRequests.Core	Модели, база данных, сервисы, экспорт	net10.0
RepairRequests.WinForms	Формы и оформление интерфейса	net10.0-windows
RepairRequests.Tests	Модульные и интеграционные тесты	net10.0

3. Полученный результат

Создано и настроено решение из трёх проектов, проверена сборка и запуск приложения. Структура каталогов соответствует слоям приложения.

4. Вывод

Разделение на проекты позволило отделить бизнес-логику от интерфейса и подготовить основу для автоматического тестирования.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 11. Настройка Git-репозитория

1. Цель и задачи этапа

Цель этапа — Настроить локальный репозиторий Git и подготовить первый коммит проекта.

Для достижения цели решались следующие задачи:

1. инициализировать репозиторий;
2. составить файл .gitignore;
3. настроить параметры работы с переводами строк;
4. выполнить первый коммит и проверить состояние.

2. Ход выполнения работ

Репозиторий инициализирован командой git init в каталоге решения. Для хранения исходного кода выбрана ветка main. Репозиторий является локальным; удалённый репозиторий не использовался, поскольку проект учебный и ведётся одним разработчиком.

В файл .gitignore включены каталоги bin и obj, скрытая папка .vs, каталог публикации publish, журналы logs, файлы базы данных с расширениями .db, .db-shm и .db-wal, а также временные каталоги, создаваемые при экспорте документов. Исключение базы данных важно: она содержит тестовые данные и создаётся заново при первом запуске приложения.

Параметр core.autocrlf установлен в значение true, чтобы переводы строк приводились к формату Windows при извлечении файлов. Для текстовых файлов исходного кода это избавляет от лишних изменений при сравнении версий.

Первым коммитом зафиксирован каркас решения: проекты, модели, контекст базы данных, миграция InitialCreate, тесты и файл .gitignore. Сообщение коммита — «Каркас решения: Core, WinForms, тесты, миграция

InitialCreate». Хеш коммита 83bb374. После коммита команда git status показывает отсутствие изменений в отслеживаемых файлах.

Таблица 1 — Основные записи файла .gitignore

Шаблон	Назначение
bin/, obj/	Результаты сборки
.vs/	Настройки среды разработки
publish/	Готовая сборка приложения
*.db, *.db-shm, *.db-wal	Файлы базы данных SQLite
logs/	Журналы работы приложения

3. Полученный результат

Репозиторий Git настроен, лишние файлы исключены, первый коммит создан. История изменений позволяет вернуться к исходному состоянию проекта.

4. Вывод

Использование Git обеспечило сохранность версий и возможность фиксировать результаты каждого этапа разработки.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 12. Создание базы данных SQLite и миграции

1. Цель и задачи этапа

Цель этапа — Создать базу данных SQLite, применить миграцию и заполнить её тестовыми данными для проверки работы системы.

Для достижения цели решались следующие задачи:

1. описать контекст базы данных и наборы сущностей;
2. создать миграцию InitialCreate;
3. реализовать заполнение базы тестовыми данными;
4. проверить структуру и объём данных.

2. Ход выполнения работ

Для хранения данных выбрана встраиваемая база SQLite. Файл базы данных располагается в каталоге data рядом с исполняемым файлом. Такой вариант не требует установки сервера баз данных и удобен для настольного учебного приложения. Путь к файлу задаётся в файле appsettings.json и может быть изменён без пересборки программы.

Контекст AppDbContext содержит наборы сущностей Users, Vehicles, FaultTypes, RepairRequests и RequestStatusHistory. В методе OnModelCreating описаны уникальные индексы, ограничения длины строк, точность денежных полей и поведение внешних ключей при удалении. Миграция InitialCreate создана командой dotnet ef migrations add и применяется автоматически при запуске приложения методом Database.Migrate().

Заполнение базы выполняет класс DbSeeder. Для воспроизводимости используется генератор случайных чисел с фиксированным начальным значением 2026, поэтому при удалении файла базы данных создаётся тот же набор записей. Добавляются шесть пользователей, семь автомобилей, десять видов неисправностей, сорок две заявки и сто восемьдесят две записи истории статусов.

Заявки распределены по всем статусам жизненного цикла, часть заявок имеет заполненные даты начала и завершения ремонта. Это позволяет проверить расчёт статистики и построение отчётов на реальных данных. Повторный запуск приложения не создаёт дубликатов: перед заполнением проверяется наличие пользователей.

Таблица 1 — Объём тестовых данных

Таблица	Количество записей
Users	6
Vehicles	7
FaultTypes	10
RepairRequests	42
RequestStatusHistory	182

3. Полученный результат

База данных создана, миграция применена, тестовые данные загружены. Объём данных достаточен для проверки всех форм и отчётов.

4. Вывод

Автоматическое применение миграции и детерминированное заполнение упростили развёртывание и сделали результаты проверки воспроизводимыми.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 13. Модель User и таблица Users

1. Цель и задачи этапа

Цель этапа — Создать модель пользователя и организовать безопасное хранение паролей.

Для достижения цели решались следующие задачи:

1. описать поля и ограничения модели User;
2. реализовать хеширование пароля;
3. предусмотреть проверку пароля без раскрытия хеша;
4. добавить служебные поля учётной записи.

2. Ход выполнения работ

Модель User содержит идентификатор, ФИО, электронную почту, хеш пароля, роль, телефон, признак блокировки, дату регистрации и дату последнего входа. Для полей заданы атрибуты валидации: ФИО от 3 до 120 символов, корректный адрес электронной почты, обязательность основных полей. Электронная почта имеет уникальный индекс.

Пароль в открытом виде не сохраняется. Класс PasswordHasher реализует алгоритм PBKDF2-HMAC-SHA256 со случайной солью длиной 16 байт, 120 000 итераций и ключом длиной 32 байта. В базе данных хранится строка формата pbkdf2-sha256, в которой указаны число итераций, соль и хеш в кодировке Base64. Соль уникальна для каждого пользователя, поэтому одинаковые пароли дают разные хеши.

Проверка пароля выполняется путём повторного вычисления хеша с сохранённой солью и числом итераций. Сравнение выполняется методом FixedTimeEquals, что исключает зависимость времени сравнения от количества совпавших байтов. При неверном формате сохранённой строки проверка завершается неуспешно без исключения.

Служебные поля IsBlocked и LastLoginAt позволяют заблокировать учётную запись и отобразить сведения о последнем входе. Роль хранится в

виде строки, что делает содержимое таблицы читаемым при просмотре базы данных и упрощает отладку.

Листинг 1 — Проверка пароля

```
var actual = Rfc2898DeriveBytes.Pbkdf2(
    password, salt, iterations, HashAlgorithmName.SHA256, expected.Length);
return CryptographicOperations.FixedTimeEquals(actual, expected);
```

3. Полученный результат

Модель User создана, пароли хранятся только в виде хеша, реализованы регистронезависимый поиск по почте и проверка учётных данных.

4. Вывод

Применение PBKDF2 с солью и постоянным временем сравнения обеспечивает приемлемый уровень защиты паролей для учебного проекта.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 14. Реализация формы входа

1. Цель и задачи этапа

Цель этапа — Разработать форму авторизации с проверкой введённых данных и обработкой ошибок входа.

Для достижения цели решались следующие задачи:

1. разместить поля ввода и кнопки;
2. выполнить проверку заполнения полей;
3. связать форму со службой авторизации;
4. предусмотреть сообщения об ошибках и переход в главное меню.

2. Ход выполнения работ

Форма входа содержит поля электронной почты и пароля, кнопку входа, кнопку перехода к регистрации и кнопку выхода из приложения. Пароль скрывается символами, рядом расположен флажок «показать пароль». Кнопка входа назначена кнопкой по умолчанию, поэтому вход выполняется нажатием клавиши Enter.

При нажатии кнопки выполняется проверка заполнения полей. Пустые значения не передаются в службу авторизации: пользователь получает сообщение о необходимости заполнить оба поля. Введённая почта приводится к нижнему регистру и обрезается от пробелов, чтобы исключить ошибки ввода.

Служба AuthService находит пользователя по почте и проверяет пароль методом PasswordHasher.Verify. При неверной почте или пароле выводится одинаковое сообщение «Неверная электронная почта или пароль», чтобы не раскрывать, какая именно часть данных не совпала. Для заблокированной учётной записи выводится отдельное сообщение с рекомендацией обратиться к администратору.

При успешном входе обновляется дата последнего входа и открывается главное меню. Форма входа скрывается, а после выхода из главного меню

снова отображается. Закрытие формы входа завершает работу приложения. Ошибки доступа к базе данных перехватываются и записываются в журнал.

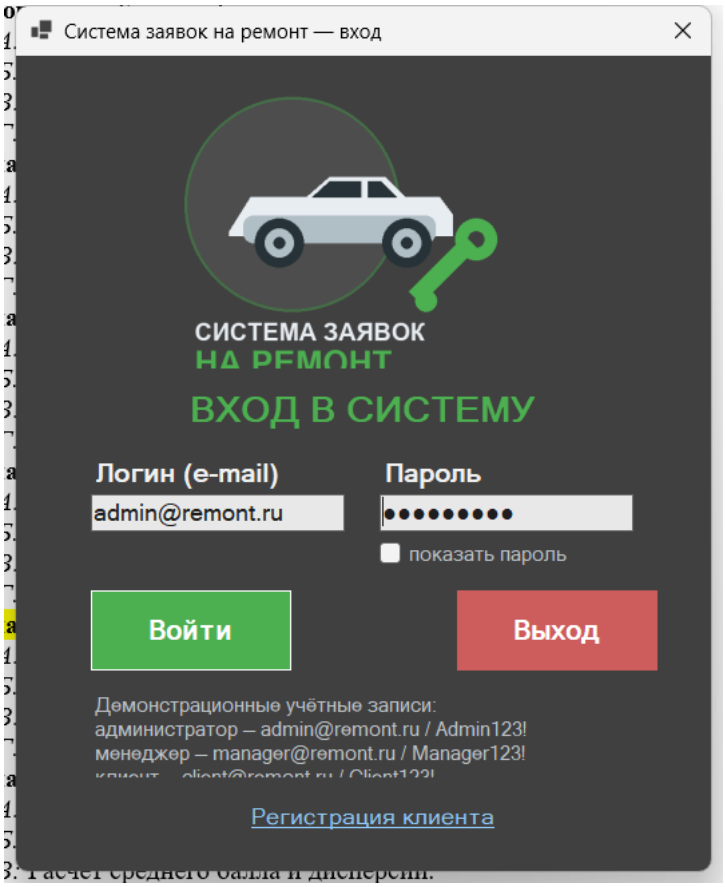


Рисунок 1 — Форма входа в систему

3. Полученный результат

Форма входа работает, ошибки обрабатываются, переход в главное меню и выход из системы выполняются корректно.

4. Вывод

Форма входа обеспечивает проверку учётных данных и служит точкой разграничения доступа к функциям системы.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 15. Разграничение ролей пользователей

1. Цель и задачи этапа

Цель этапа — Реализовать разграничение прав доступа для клиента, менеджера и администратора.

Для достижения цели решались следующие задачи:

1. описать перечисление ролей;
2. ограничить видимость данных в запросах;
3. настроить состав кнопок главного меню;
4. проверить работу под каждой ролью.

2. Ход выполнения работ

Роли описаны перечислением UserRole со значениями Client, Manager и Administrator. Роль хранится в учётной записи и загружается при входе. Неавторизованный посетитель не имеет роли и может работать только с формой входа и регистрации.

Клиент видит в главном меню разделы «Мои заявки», «Мой транспорт», «Отчёт» и «Выход». В списке заявок и автомобилей применяется фильтр по идентификатору текущего пользователя, поэтому клиент не может увидеть чужие записи даже при изменении параметров фильтра. При открытии чужой заявки по идентификатору выполняется дополнительная проверка владельца.

Менеджер видит все заявки и автомобили, а также раздел «Неисправности». Ему доступны приём заявки, назначение механика, изменение статуса и формирование отчёта за произвольный период. Роль администратора в учебной версии пользуется теми же разделами, что и менеджер; отдельная форма управления пользователями не предусмотрена, поскольку блокировка учётных записей реализована на уровне модели данных.

Проверка выполнялась последовательным входом под учётными записями клиента, менеджера и администратора. Для клиента дополнительно проверено, что в выпадающих списках автомобилей и заявок отсутствуют чужие записи. Результаты проверки совпадают с описанием ролей в техническом задании.

Таблица 1 — Права пользователей по ролям

Раздел	Клиент	Менеджер	Администратор
Свои заявки	Просмотр и создание	Все заявки	Все заявки
Автомобили	Только свои	Все	Все
Неисправности	Нет доступа	Просмотр и изменение	Просмотр и изменение
Отчёт	По своим заявкам	За период	За период

3. Полученный результат

Разграничение ролей реализовано на уровне интерфейса и запросов к базе данных. Клиент работает только со своими данными, сотрудник — со всеми.

4. Вывод

Ограничение выборки на уровне репозитория исключает доступ клиента к чужим заявкам и повышает защищённость данных.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 16. Справочник транспортных средств и неисправностей

1. Цель и задачи этапа

Цель этапа — Создать модели справочников транспортных средств и неисправностей, определить их поля и ограничения.

Для достижения цели решались следующие задачи:

1. описать модель Vehicle;
2. описать модель FaultType;
3. связать справочники с заявками и пользователями;
4. проверить ограничения на тестовых данных.

2. Ход выполнения работ

Модель Vehicle содержит владельца, марку, модель, государственный номер, VIN, год выпуска, пробег и дату добавления. Владелец выбирается из числа пользователей с ролью клиента. Государственный номер проверяется регулярным выражением и имеет уникальный индекс, VIN должен содержать ровно 17 символов без букв I, O и Q.

Модель FaultType описывает вид неисправности: наименование, категорию, описание, норматив трудоёмкости и базовую стоимость работ. Категории объединяют неисправности по системам автомобиля: двигатель, тормозная система, подвеска, кузов, электрооборудование и другие. Норматив используется при планировании и при расчёте предварительной стоимости.

Справочники связаны с заявками отношениями один ко многим. Автомобиль может участвовать в нескольких заявках, неисправность может быть указана в разных заявках. Удаление записей, на которые ссылаются заявки, запрещено. Для устаревших неисправностей предусмотрен признак активности: неактивная запись остаётся в истории, но не предлагается при оформлении новой заявки.

Ограничения проверены на тестовых данных: попытка добавить автомобиль с некорректным номером или повторяющимся VIN приводит к сообщению об ошибке. Справочник неисправностей заполнен десятью записями с различными категориями и нормативами от 0,5 до 8 часов.

Таблица 1 — Основные поля справочников

Модель	Поле	Ограничение
Vehicle	LicensePlate	Уникальный, формат A123BC77
Vehicle	Vin	17 символов, без I, O, Q
Vehicle	Year	1950–2100
FaultType	Name	3–120 символов
FaultType	NormHours	0,1–200 часов

3. Полученный результат

Справочники созданы и связаны с заявками. Валидация полей и уникальность ключевых значений проверены.

4. Вывод

Справочники образуют основу для оформления заявок и обеспечивают единообразие описания автомобилей и неисправностей.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 17. Разработка формы справочника транспортных средств

1. Цель и задачи этапа

Цель этапа — Реализовать форму просмотра, добавления, изменения и удаления транспортных средств.

Для достижения цели решались следующие задачи:

1. вывести список записей в таблицу;
2. реализовать добавление и редактирование;
3. предусмотреть подтверждение удаления;
4. ограничить набор данных в зависимости от роли.

2. Ход выполнения работ

Форма справочника содержит таблицу с колонками «Марка», «Модель», «Госномер», «Год», «Пробег» и «Владелец». Данные загружаются асинхронно, чтобы интерфейс не замирал при обращении к базе. В нижней части окна отображается количество показанных и общее число записей.

Добавление и редактирование выполняются в отдельной форме VehicleEditForm. При добавлении владелец по умолчанию совпадает с текущим клиентом, при работе менеджера владельца можно выбрать из списка клиентов. После сохранения список обновляется, выделяется изменённая строка.

Удаление выполняется только после подтверждения. Если на автомобиль ссылаются заявки, база данных запрещает удаление, а приложение выводит понятное сообщение и предлагает сначала закрыть или удалить связанные заявки. Для клиента форма показывает только его автомобили, для менеджера и администратора — все.

Ошибки сохранения обрабатываются: при некорректном госномере или VIN форма остаётся открытой и показывает сообщение рядом с полем. После успешного сохранения запись отображается в общем списке без повторной загрузки приложения.

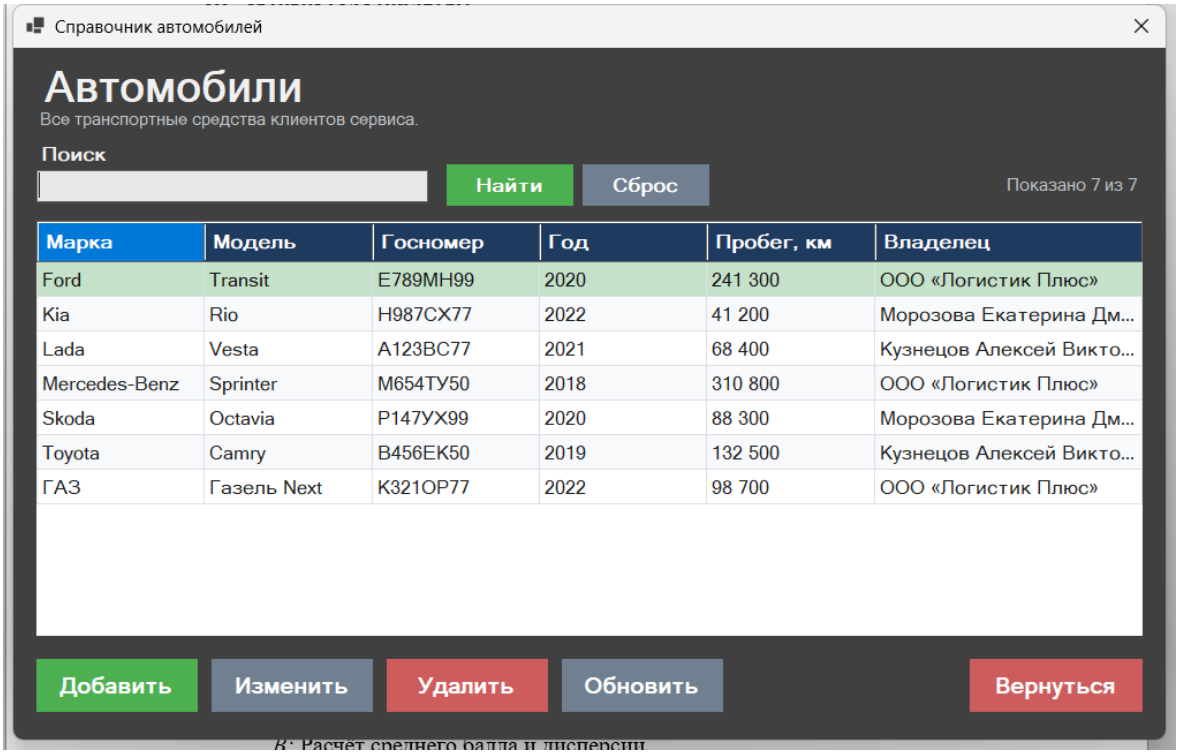


Рисунок 1 — Справочник транспортных средств

3. Полученный результат

Форма справочника работает в режимах просмотра, добавления, изменения и удаления. Разграничение данных по ролям выполнено.

4. Вывод

Форма справочника обеспечивает ведение нормативно-справочной информации и служит источником данных для оформления заявок.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 18. Поиск и валидация в справочнике транспортных средств

1. Цель и задачи этапа

Цель этапа — Реализовать поиск по нескольким полям и проверку корректности вводимых данных.

Для достижения цели решались следующие задачи:

1. добавить строку поиска;
2. реализовать фильтрацию по марке, модели, номеру и владельцу;
3. настроить проверку полей формы;
4. проверить обработку ошибочных значений.

2. Ход выполнения работ

В форме справочника предусмотрена строка поиска с подсказкой «марка, модель, номер, владелец». Поиск выполняется после нажатия клавиши Enter и не требует перезагрузки формы. Кнопка сброса очищает строку и возвращает полный список.

Фильтрация выполняется в репозитории: строка поиска сравнивается с маркой, моделью, государственным номером и ФИО владельца. Поиск не чувствителен к регистру, для частичного совпадения используется конструкция LIKE. Условия объединяются логическим ИЛИ, поэтому достаточно ввести часть номера или фамилию владельца.

Проверка полей выполняется как на уровне модели, так и в форме. Марка и модель не могут быть пустыми, государственный номер должен соответствовать формату A123BC77, VIN — содержать 17 допустимых символов, год выпуска — находиться в диапазоне 1950–2100, пробег — быть неотрицательным. При ошибке сохранение прекращается, поле подсвечивается, выводится текстовое пояснение.

Отдельно проверена обработка повторяющегося государственного номера: база данных возвращает ошибку уникальности, приложение

переводит её в сообщение «Автомобиль с таким номером уже добавлен». Проверка выполнена на наборе ошибочных значений, все случаи обработаны без аварийного завершения программы.

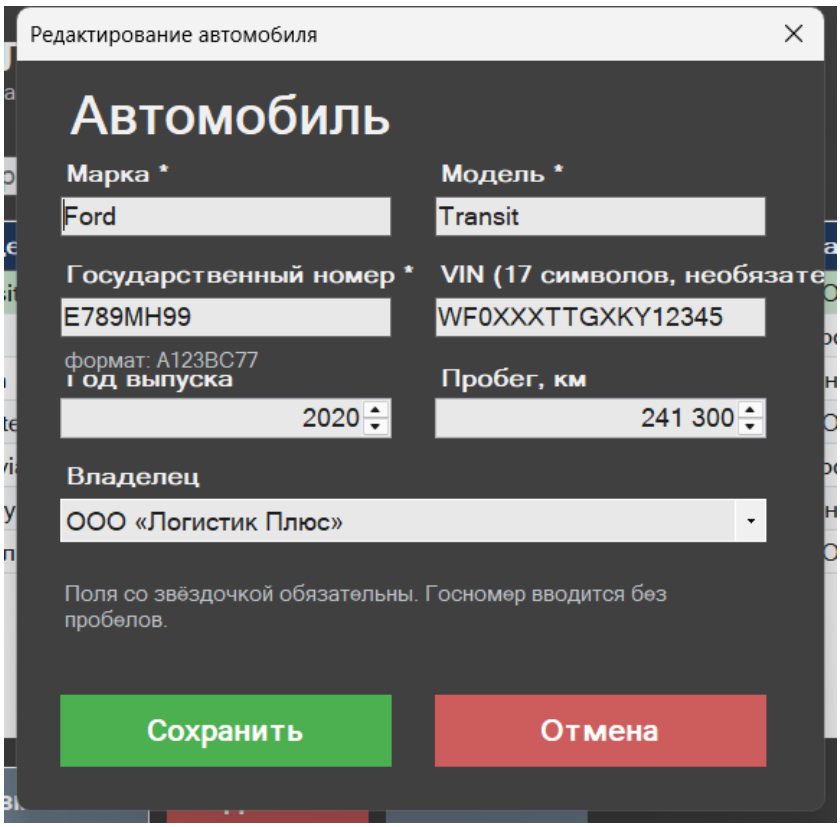


Рисунок 1 — Форма добавления автомобиля

3. Полученный результат

Поиск работает по четырём полям, валидация предотвращает сохранение некорректных данных. Ошибочные ситуации обрабатываются понятными сообщениями.

4. Вывод

Поиск и проверка ввода повышают удобство работы со справочником и снижают количество ошибок в данных.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 19. Модель заявки на ремонт и история статусов

1. Цель и задачи этапа

Цель этапа — Создать модель заявки на ремонт и модель истории изменения статусов.

Для достижения цели решались следующие задачи:

1. описать состав полей заявки;
2. определить перечисление статусов и приоритетов;
3. связать заявку с автомобилем, клиентом и механиком;
4. реализовать вычисляемые показатели длительности.

2. Ход выполнения работ

Модель RepairRequest является центральной сущностью системы. Она содержит номер заявки, ссылки на автомобиль, неисправность, клиента и механика, описание неисправности, приоритет, статус, плановую дату завершения, стоимости работ и запчастей, комментарий механика и набор дат жизненного цикла.

Жизненный цикл описан перечислением RequestStatus: New, Accepted, InProgress, Completed, Closed и Cancelled. Дополнительно задано перечисление RequestPriority с уровнями Low, Normal, High и Critical. Для отображения в интерфейсе созданы методы расширения, возвращающие русские названия статусов и приоритетов.

Даты CreatedAt, AcceptedAt, StartedAt, CompletedAt и ClosedAt фиксируют моменты перехода заявки между состояниями. На их основе вычисляются длительность ремонта от начала работ до выполнения, время ожидания в очереди и полный простой от создания заявки до завершения. Эти величины не хранятся в базе, а рассчитываются в модели, что исключает расхождение данных.

История изменения статусов вынесена в модель RequestStatusHistory. Каждая запись содержит ссылку на заявку, новый статус, дату изменения,

автора и комментарий. История позволяет восстановить последовательность обработки заявки и используется при проверке корректности расчёта времени ремонта.

Таблица 1 — Статусы заявки

Статус	Название	Смысл
New	Новая	Заявка создана клиентом или менеджером
Accepted	Принята	Менеджер принял заявку в работу
InProgress	В работе	Механик выполняет ремонт
Completed	Выполнена	Работы завершены
Closed	Закрыта	Заявка закрыта после выдачи автомобиля
Cancelled	Отменена	Заявка отменена по инициативе клиента или станции

3. Полученный результат

Модели заявки и истории статусов созданы, определены связи и вычисляемые показатели длительности. Статусы соответствуют принятому процессу ремонта.

4. Вывод

Модель заявки объединяет сведения о клиенте, автомобиле, работах и сроках и служит основой для статистической обработки.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 20. Форма регистрации заявки на ремонт

1. Цель и задачи этапа

Цель этапа — Разработать форму создания и редактирования заявки с проверкой введённых данных.

Для достижения цели решались следующие задачи:

1. разместить поля формы и элементы выбора;
2. реализовать автоматическую нумерацию заявок;
3. выполнить проверку обязательных полей;
4. сохранить заявку и отобразить результат.

2. Ход выполнения работ

Форма RequestEditForm используется для создания новой заявки и для просмотра или изменения существующей. В верхней части отображается номер заявки и текущий статус, ниже расположены списки выбора автомобиля и неисправности, поле описания, приоритет, плановая дата завершения, стоимость работ и стоимость запчастей.

Номер заявки формируется автоматически в формате ЗР-2026-0001. Репозиторий находит последний номер за текущий год и увеличивает порядковую часть на единицу. Нумерация не зависит от идентификатора записи, поэтому номер остаётся понятным при просмотре отчёта и в общении с клиентом.

Проверка полей выполняется перед сохранением. Автомобиль и неисправность должны быть выбраны, описание — содержать от 10 до 1000 символов, стоимости — быть неотрицательными. При ошибке форма не закрывается, пользователь получает сообщение и может исправить данные. Кнопка сохранения блокируется на время записи, чтобы исключить повторное нажатие.

Для существующей заявки форма показывает историю статусов в отдельной таблице. Кнопки смены статуса доступны только сотруднику и

только для допустимых переходов. Клиент видит заявку в режиме просмотра, если она принадлежит ему.

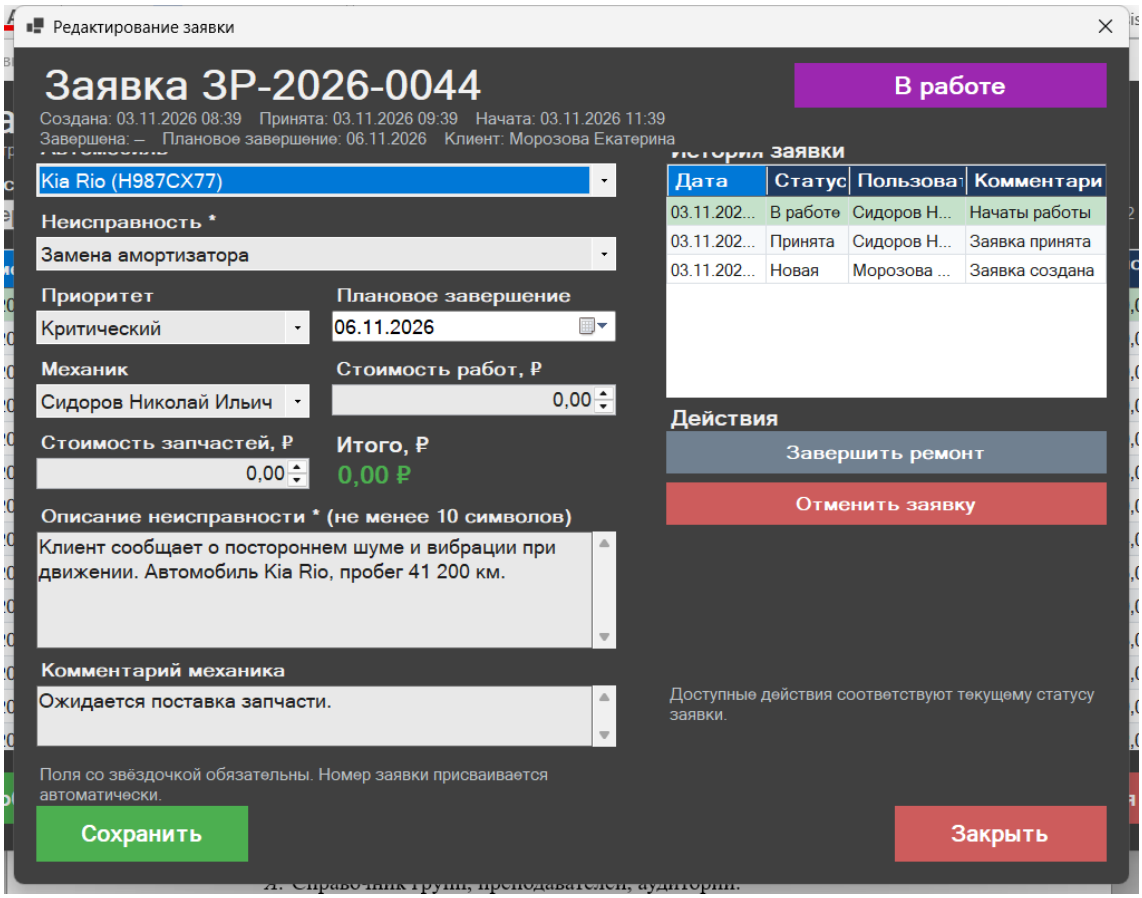


Рисунок 1 — Форма заявки на ремонт

3. Полученный результат

Форма регистрации заявки работает, номера присваиваются автоматически, проверка полей предотвращает сохранение неполных данных.

4. Вывод

Форма обеспечивает единый порядок оформления заявок и фиксирует все сведения, необходимые для выполнения работ и расчёта статистики.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 21. Интеграция формы заявки со справочниками

1. Цель и задачи этапа

Цель этапа — Организовать выбор автомобиля и неисправности из справочников при оформлении заявки.

Для достижения цели решались следующие задачи:

1. заполнить списки выбора данными справочников;
2. ограничить список автомобилей правами клиента;
3. автоматически подставлять норматив и стоимость;
4. проверить сохранение связей в базе данных.

2. Ход выполнения работ

Списки выбора автомобилей и неисправностей заполняются при открытии формы. Для автомобилей используется метод получения всех доступных записей с учётом области видимости: клиент получает только свои автомобили, сотрудник — все. В списке отображается марка, модель и государственный номер.

Список неисправностей содержит только активные записи. При выборе неисправности форма автоматически подставляет базовую стоимость работ и показывает норматив трудоёмкости. Это сокращает ручной ввод и снижает вероятность завышения или занижения стоимости по сравнению со справочником.

При сохранении в заявку записываются идентификаторы выбранных записей, а не их текстовые названия. Благодаря этому изменение наименования неисправности или марки автомобиля не приводит к рассогласованию данных. Связи контролируются внешними ключами базы данных.

Проверено оформление заявки клиентом и менеджером. Клиент не видит в списке чужие автомобили, менеджер может оформить заявку на

любой автомобиль. После сохранения заявка появляется в общем списке и открывается для просмотра.

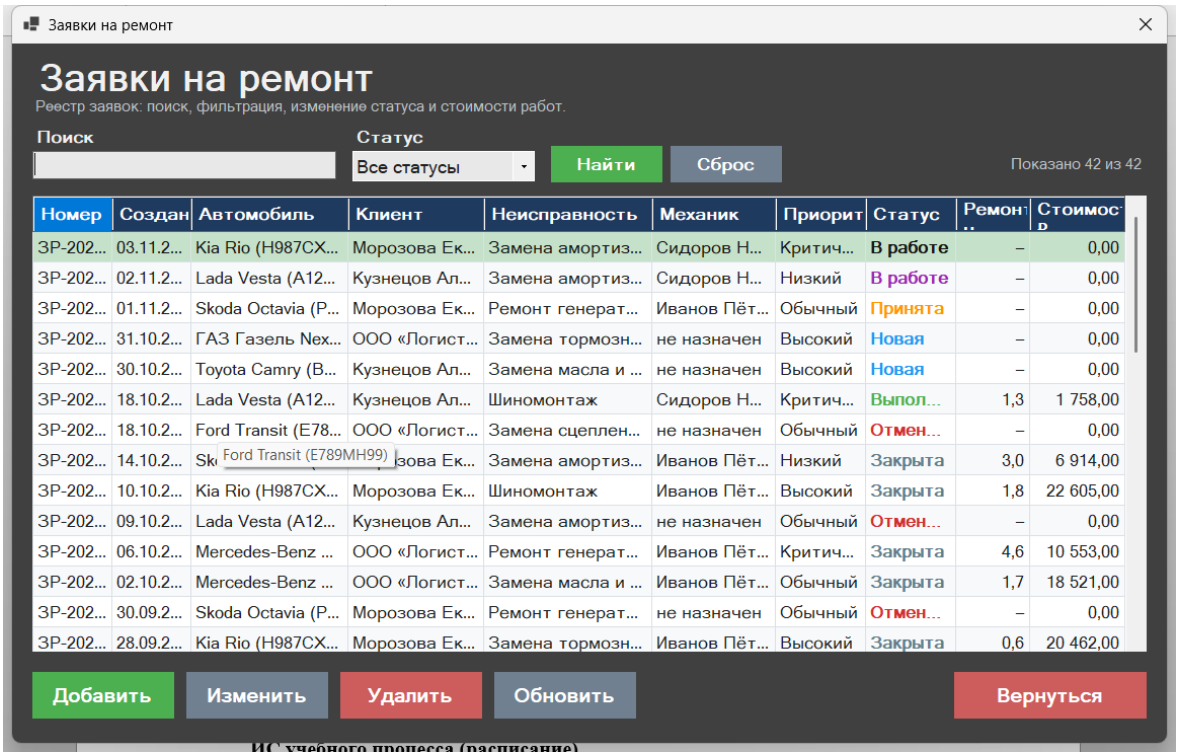


Рисунок 1 — Список заявок на ремонт

3. Полученный результат

Выбор данных из справочников работает, список автомобилей ограничен правами пользователя, значения подставляются автоматически.

4. Вывод

Интеграция формы со справочниками обеспечивает корректные связи и сокращает объём ручного ввода при оформлении заявки.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 22. Слой доступа к данным

1. Цель и задачи этапа

Цель этапа — Реализовать репозитории для работы с заявками, автомобилями, неисправностями и пользователями.

Для достижения цели решались следующие задачи:

1. описать интерфейсы репозиториев;
2. реализовать основные операции чтения и записи;
3. вынести построение запросов из форм;
4. проверить работу репозиториев в тестах.

2. Ход выполнения работ

Доступ к данным организован через репозитории. Интерфейсы IUserRepository, IVehicleRepository, IFaultTypeRepository и IRepairRequestRepository описывают операции получения списка, поиска по идентификатору, добавления, изменения и удаления. Формы работают с интерфейсами и не содержат запросов к базе данных напрямую.

Репозиторий заявок реализует выборку с фильтрацией, сортировкой и постраничным выводом. Запросы строятся постепенно: сначала применяются условия отбора, затем сортировка, после чего вычисляется общее количество записей и выбирается нужная страница. Для чтения списков используется режим без отслеживания изменений, что снижает нагрузку на контекст.

Для заявок реализованы дополнительные операции: получение заявок клиента, получение активных заявок, генерация номера, добавление записи истории и изменение статуса. Операции, затрагивающие несколько таблиц, выполняются в одной транзакции, чтобы не допустить частичного сохранения.

Каждый экземпляр контекста создаётся через фабрику и освобождается после завершения операции. Такой подход соответствует рекомендациям

Entity Framework Core для настольных приложений и исключает накопление отслеживаемых сущностей при длительной работе программы.

Таблица 1 — Основные репозитории

Репозиторий	Основные операции
UserRepository	Поиск по почте, проверка существования, обновление
VehicleRepository	Список с поиском, получение по владельцу
FaultTypeRepository	Список с фильтром, активные неисправности
RepairRequestRepository	Фильтрация, сортировка, страницы, история

3. Полученный результат

Репозитории реализованы, формы используют интерфейсы, запросы вынесены из интерфейсного слоя. Работа репозиториев проверена интеграционными тестами.

4. Вывод

Слой доступа к данным упростил сопровождение приложения и позволил тестировать выборки без запуска форм.

Обучающийся: ________ / _____

Дата: «____» _____ 2026 г.

Этап 23. Запросы с фильтрацией и сортировкой

1. Цель и задачи этапа

Цель этапа — Реализовать выборку заявок по различным условиям и настраиваемую сортировку.

Для достижения цели решались следующие задачи:

1. описать параметры фильтра;
2. реализовать отбор по статусу, приоритету и автомобилю;
3. добавить сортировку по дате, номеру и статусу;
4. проверить результаты запросов на тестовых данных.

2. Ход выполнения работ

Параметры выборки описаны классом RequestFilter. Он содержит строку поиска, статус, приоритет, идентификаторы автомобиля, неисправности и клиента, диапазон дат, способ сортировки и номер страницы. Значения параметров необязательны: если поле не задано, условие не добавляется в запрос.

Строка поиска сравнивается с номером заявки, маркой и моделью автомобиля, фамилией клиента и наименованием неисправности. Отбор по статусу и приоритету выполняется по перечислению, отбор по автомобилю и клиенту — по внешнему ключу. Диапазон дат задаётся двумя границами и применяется к дате создания заявки.

Сортировка поддерживает несколько режимов: по дате создания по убыванию и возрастанию, по номеру, по статусу и по приоритету. По умолчанию используется сортировка по дате по убыванию, чтобы новые заявки отображались первыми. При равных значениях применяется дополнительный порядок по номеру.

Проверка выполнялась на наборе из сорока двух заявок. Сверены количества записей при отборе по каждому статусу, проверены границы

диапазона дат и порядок сортировки. Результаты запросов совпадают с данными, отображаемыми в формах и отчёте.

Таблица 1 — Режимы сортировки заявок

Значение	Порядок
date_desc	Дата создания по убыванию
date_asc	Дата создания по возрастанию
number	Номер заявки
status	Статус заявки
priority	Приоритет заявки

3. Полученный результат

Фильтрация и сортировка работают, параметры выборки вынесены в отдельный класс, запросы проверены на реальных данных.

4. Вывод

Единый механизм фильтрации используется и в списке заявок, и в отчёте, что обеспечивает одинаковые результаты в разных разделах.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 24. Обеспечение ссылочной целостности при работе с заявками

1. Цель и задачи этапа

Цель этапа — Проверить и обеспечить сохранность связанных данных при изменении и удалении заявок.

Для достижения цели решались следующие задачи:

1. настроить поведение внешних ключей;
2. предусмотреть обработку ошибок удаления;
3. сохранять историю статусов вместе с заявкой;
4. проверить сценарии на тестовых данных.

2. Ход выполнения работ

При удалении заявки связанные записи истории удаляются каскадно, поскольку история не имеет самостоятельного значения без заявки. Для справочных данных используется запрет удаления: нельзя удалить автомобиль, неисправность или пользователя, если на них ссылаются заявки.

В приложении обрабатываются исключения базы данных, возникающие при нарушении ограничений. Вместо технического текста ошибки пользователь видит сообщение о том, что запись используется и не может быть удалена. Для неисправностей предлагается сделать запись неактивной.

Изменение статуса заявки и добавление записи истории выполняются в одной транзакции. Если запись истории не сохранилась, изменение статуса также отменяется. Это исключает ситуацию, когда заявка перешла в новый статус, но сведений об этом в истории нет.

Проверены сценарии удаления автомобиля с заявками, удаления неисправности, используемой в заявках, и удаления заявки с историей. Во всех случаях данные остались согласованными, а приложение вывело понятные сообщения.

3. Полученный результат

Ограничения ссылочной целостности настроены и проверены. История статусов сохраняется и удаляется вместе с заявкой.

4. Вывод

Правила целостности защищают историю ремонтов от случайного удаления справочных данных.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 25. Фильтрация заявок по диапазону дат

1. Цель и задачи этапа

Цель этапа — Реализовать отбор заявок за заданный период и проверить граничные случаи.

Для достижения цели решались следующие задачи:

1. добавить параметры начала и окончания периода;
2. учесть время суток при сравнении дат;
3. применить фильтр в отчёте;
4. проверить пустой и обратный период.

2. Ход выполнения работ

В параметрах выборки предусмотрены поля DateFrom и DateTo. Если задана дата начала, в выборку попадают заявки, созданные не раньше этого дня; если задана дата окончания — не позже конца указанного дня. Время суток при сравнении учитывается: дата окончания приводится к 23:59:59, иначе заявки последнего дня могли бы не попасть в отчёт.

В форме отчёта период задаётся двумя элементами выбора даты. По умолчанию устанавливается начало текущего месяца за пять месяцев до текущей даты, что позволяет сразу увидеть динамику за полугодие. Перед формированием отчёта проверяется, что начало периода не позже окончания.

Фильтр применяется к запросу до вычисления статистики, поэтому все показатели отчёта рассчитываются только по заявкам выбранного периода. Для отчёта используется тот же репозиторий, что и для списка заявок, но с обязательными границами периода.

Проверены три случая: обычный период с данными, период без заявок и период с обратным порядком дат. В первом случае отчёт формируется, во втором выводятся нулевые показатели и сообщение об отсутствии данных, в третьем выводится предупреждение и отчёт не строится.

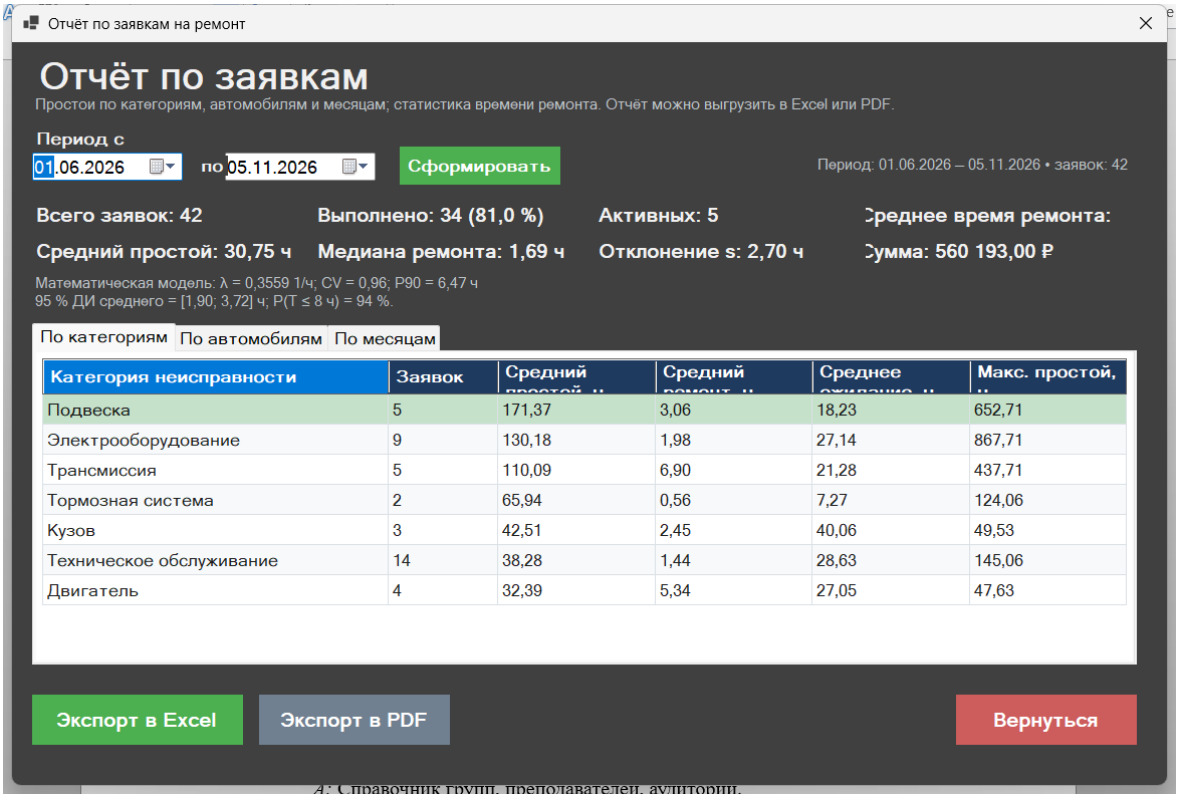


Рисунок 1 — Отчёт за выбранный период

3. Полученный результат

Отбор по диапазону дат работает, границы периода учитываются корректно, пустой период обрабатывается без ошибок.

4. Вывод

Фильтр по датам позволяет формировать отчёт за нужный период и является основой для анализа динамики ремонтов.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 26. Фильтрация заявок по клиенту

1. Цель и задачи этапа

Цель этапа — Реализовать отбор заявок конкретного клиента и ограничение видимости данных по роли.

Для достижения цели решались следующие задачи:

1. добавить параметр идентификатора клиента;
2. применить ограничение для роли клиента;
3. проверить отсутствие доступа к чужим заявкам;
4. отобразить клиента в списке и отчёте.

2. Ход выполнения работ

Параметр ClientId позволяет выбрать заявки конкретного клиента. Он используется в двух случаях: когда сотрудник хочет посмотреть историю обслуживания определённого владельца и когда клиент открывает раздел «Мои заявки».

Для роли клиента ограничение применяется автоматически. В контексте приложения хранится текущий пользователь, а свойство ClientScope возвращает его идентификатор для клиента и пустое значение для сотрудника. Формы и отчёт передают эту область видимости в репозиторий, поэтому клиент физически не получает чужие записи из базы данных.

В списке заявок отображается колонка «Клиент», в отчёте предусмотрена группировка по клиентам с количеством заявок и средним временем ремонта. Это позволяет сотруднику оценить, какие клиенты обращаются чаще и по каким работам.

Проверка выполнялась под учётными записями трёх клиентов. Каждый клиент видел только свои автомобили и заявки, количество записей совпадало с данными в базе. Попытка открыть заявку другого клиента по идентификатору завершилась отказом.

3. Полученный результат

Отбор по клиенту реализован, ограничение видимости применяется на уровне запросов и проверено под разными учётными записями.

4. Вывод

Фильтрация по клиенту обеспечивает как удобство работы сотрудника, так и защиту персональных данных клиентов.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 27. Сортировка и постраничный вывод заявок

1. Цель и задачи этапа

Цель этапа — Реализовать постраничную выборку заявок и сохранение порядка сортировки при переходе между страницами.

Для достижения цели решались следующие задачи:

1. описать результат постраничной выборки;
2. вычислить общее количество страниц;
3. передавать номер страницы и размер страницы;
4. проверить границы первой и последней страницы.

2. Ход выполнения работ

Постраничная выборка описана классом PagedResult, который содержит список записей текущей страницы, общее количество записей, номер страницы и размер страницы. На основе этих данных вычисляются общее количество страниц и признаки наличия предыдущей и следующей страницы.

Репозиторий сначала подсчитывает общее количество записей по условиям фильтра, затем применяет сортировку и выбирает только нужный диапазон с помощью методов Skip и Take. Такой порядок позволяет не загружать в память весь список при большом объёме данных.

В формах списка используется размер страницы 500 записей, поскольку учебный набор данных невелик и пользователю удобнее видеть все заявки сразу. При увеличении объёма данных достаточно уменьшить размер страницы и добавить кнопки перехода, не изменяя запросы репозитория.

Проверены первая, промежуточная и последняя страницы при размере 10 записей. Количество записей на странице соответствует ожидаемому, общее количество совпадает с числом заявок в базе, порядок сортировки сохраняется при переходе между страницами.

Таблица 1 — Состав результата постраничной выборки

Свойство	Назначение
Items	Записи текущей страницы
TotalCount	Общее количество записей
Page	Номер текущей страницы
PageSize	Размер страницы
TotalPages	Общее количество страниц

3. Полученный результат

Постраничная выборка реализована, границы страниц обрабатываются корректно, сортировка сохраняется.

4. Вывод

Постраничный вывод подготавливает систему к работе с большим количеством заявок без изменения структуры запросов.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 28. Постановка задачи статистики времени ремонта

1. Цель и задачи этапа

Цель этапа — Сформулировать математическую задачу расчёта среднего времени ремонта и выбрать показатели для анализа.

Для достижения цели решались следующие задачи:

1. определить генеральную совокупность и выборку;
2. выбрать показатели описательной статистики;
3. обосновать вероятностную модель;
4. записать формулы расчёта.

2. Ход выполнения работ

В качестве случайной величины рассматривается время непосредственного ремонта T — интервал от момента начала работ до момента выполнения заявки. Значения вычисляются по датам StartedAt и CompletedAt для заявок, у которых заполнены обе даты. Заявки без начала или без завершения в выборку не включаются, поскольку для них длительность ещё не определена.

Выборка формируется по завершённым и закрытым заявкам за выбранный период. Объём выборки обозначается n. Для описания данных рассчитываются среднее арифметическое, медиана, мода, дисперсия, среднее квадратическое отклонение, коэффициент вариации, минимальное и максимальное значения.

Среднее значение оценивает типичную длительность ремонта, медиана устойчива к редким длительным работам, мода показывает наиболее частый интервал. Дисперсия и стандартное отклонение характеризуют разброс, а коэффициент вариации позволяет сравнить разброс с уровнем среднего значения.

Для вероятностной оценки используется экспоненциальное распределение с интенсивностью $\lambda = 1 / \bar{x}$. Функция распределения $F(t) = 1 -$

e^(−λt) задаёт вероятность завершить ремонт не дольше чем за t часов. Дополнительно рассчитываются медиана распределения, 90-й процентиль и доверительный интервал для среднего.

Таблица 1 — Показатели статистики

Показатель	Обозначение	Смысл
Среднее	$\bar{x}$	Типичная длительность ремонта
Медиана	Me	Середина упорядоченной выборки
Дисперсия	s^2	Мера разброса значений
Отклонение	s	Разброс в часах
Вариация	CV	Отношение разброса к среднему
Интенсивность	λ	Среднее число ремонтов в час

3. Полученный результат

Сформулирована задача статистики, определены показатели и вероятностная модель. Формулы используются при программной реализации.

4. Вывод

Постановка задачи связала данные, накопленные в заявках, с математическими методами оценки времени ремонта.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 29. Реализация расчёта статистики в коде

1. Цель и задачи этапа

Цель этапа — Реализовать расчёт показателей описательной статистики и параметров экспоненциальной модели по данным из базы.

Для достижения цели решались следующие задачи:

1. подготовить и отфильтровать исходные значения;
2. рассчитать среднее, медиану и моду;
3. вычислить дисперсию и производные показатели;
4. обработать пустую выборку.

2. Ход выполнения работ

Расчёт сосредоточен в классе RepairStatisticsService. На вход поступает последовательность длительностей в часах. Значения проверяются на конечность и неотрицательность, после чего сортируются по возрастанию. Сортировка нужна для определения медианы, минимума, максимума и моды.

Среднее вычисляется как сумма значений, делённая на их количество. Медиана при нечётном количестве совпадает со средним элементом, при чётном — равна полусумме двух средних элементов. Мода определяется как середина часового интервала, в который попало наибольшее число значений.

Дисперсия рассчитывается как сумма квадратов отклонений от среднего, делённая на $n - 1$. Используется исправленная выборочная дисперсия, поскольку данные представляют выборку, а не всю генеральную совокупность. Стандартное отклонение равно квадратному корню из дисперсии, коэффициент вариации — отношению отклонения к среднему.

Параметры экспоненциальной модели вычисляются по среднему: $\lambda = 1 / \bar{x}$, медиана распределения равна $\ln(2) / \lambda$, 90-й процентиль — $-\ln(0{,}1) / \lambda$. Доверительный интервал для среднего строится по формуле $\bar{x} \pm 1{,}96 \cdot s / \sqrt{n}$. При пустой выборке возвращается объект с нулевыми значениями без исключения.

Листинг 1 — Расчёт дисперсии и интенсивности

```
var variance = n > 1
    ? data.Sum(h => Math.Pow(h - mean, 2)) / (n - 1)
    : 0;
var std = Math.Sqrt(variance);
var lambda = mean > 0 ? 1 / mean : 0;
```

3. Полученный результат

Расчёт реализован, показатели округляются до двух знаков, интенсивность — до четырёх. Пустая выборка обрабатывается корректно.

4. Вывод

Сервис статистики не зависит от интерфейса и может быть проверен модульными тестами на подготовленных наборах значений.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 30. Интеграция расчёта статистики в форму отчёта

1. Цель и задачи этапа

Цель этапа — Вывести рассчитанные показатели в форме отчёта и связать их с выбранным периодом.

Для достижения цели решались следующие задачи:

1. добавить блоки показателей;
2. связать расчёт с периодом отчёта;
3. отобразить вероятностные оценки;
4. проверить поведение при отсутствии данных.

2. Ход выполнения работ

В форме отчёта показатели выведены в верхней части в виде компактных блоков: общее число заявок, количество выполненных, число активных, среднее время ремонта, медиана, стандартное отклонение, суммарная стоимость. Значения обновляются при каждом формировании отчёта за выбранный период.

Отдельный блок посвящён математической модели. В нём отображаются интенсивность λ, вероятность завершить ремонт за два, четыре и восемь часов, вероятность того, что ремонт займёт более четырёх часов, а также 90-й процентиль времени ремонта. Формулы подписаны рядом с показателями.

Ниже расположены вкладки с таблицами: распределение по категориям неисправностей, по автомобилям и по месяцам. В каждой таблице приведены количество заявок, средний простой, среднее время ремонта, среднее ожидание и максимальный простой. Данные берутся из того же отчёта, что и верхние показатели.

Если за период нет заявок с завершённым ремонтом, статистика не рассчитывается: выводятся нули и сообщение о недостатке данных для

построения модели. Это исключает деление на ноль и появление неопределённых значений в интерфейсе.

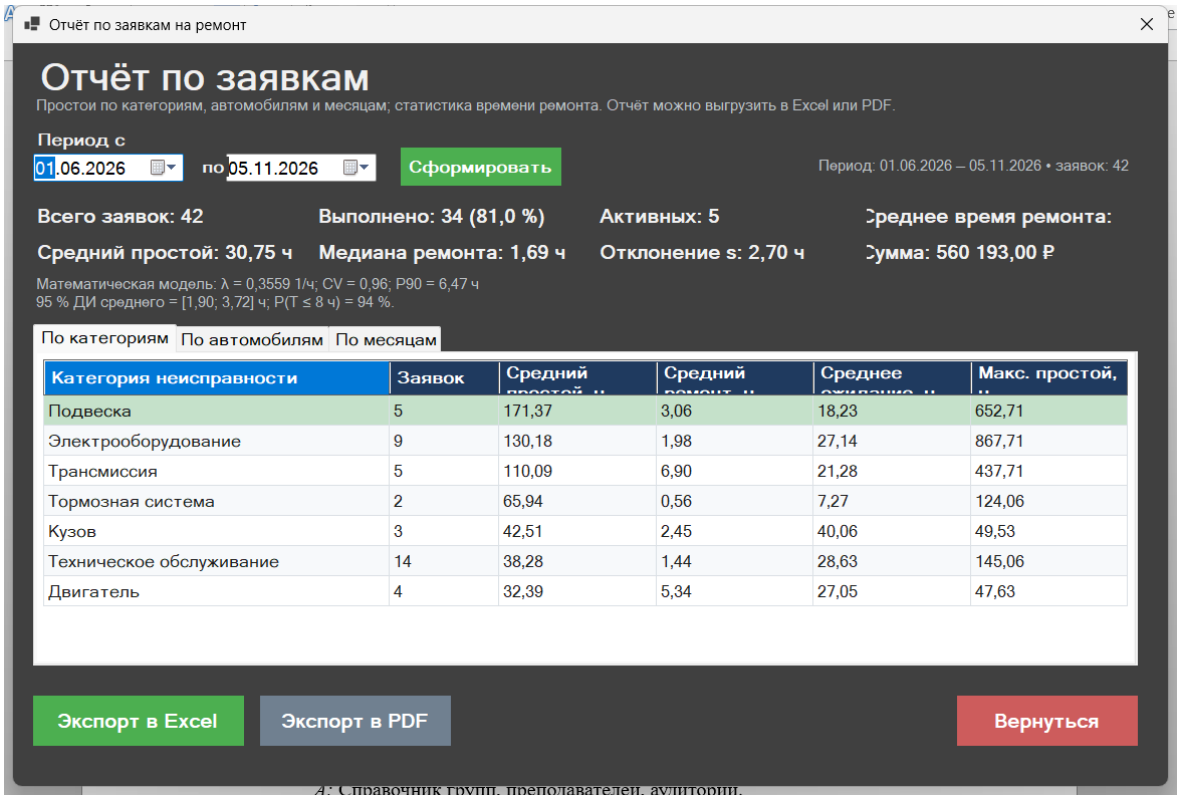


Рисунок 1 — Показатели отчёта

3. Полученный результат

Показатели статистики отображаются в форме отчёта и пересчитываются при изменении периода. Пустой период обрабатывается корректно.

4. Вывод

Интеграция статистики в отчёт позволяет пользователю получать аналитические сведения без отдельных вычислений.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 31. Вывод статистики в экспортируемых отчётах

1. Цель и задачи этапа

Цель этапа — Включить показатели статистики в отчёты Excel и PDF и проверить согласованность значений.

Для достижения цели решались следующие задачи:

1. добавить лист или раздел со статистикой;
2. перенести формулы и значения;
3. проверить совпадение с экранной формой;
4. оценить читаемость отчёта.

2. Ход выполнения работ

При выгрузке в Excel создаётся отдельный лист «Статистика». На нём приведены исходные показатели: количество наблюдений, среднее, медиана, мода, дисперсия, стандартное отклонение, коэффициент вариации, минимум и максимум. Ниже размещены параметры экспоненциальной модели и вероятности завершения ремонта.

В PDF-отчёте статистике посвящён третий раздел. Он содержит таблицу показателей с формулами и краткие пояснения, а также блок вероятностей для двух, четырёх и восьми часов. Раздел расположен после сводных показателей и распределения заявок, чтобы отчёт читался последовательно.

Значения в экспортируемых файлах формируются из того же объекта статистики, что и на экране. Поэтому расхождений между интерфейсом, Excel и PDF быть не может. Дата и время формирования, период и пользователь указываются в верхней части каждого отчёта.

Проверено формирование файлов за период с данными и за пустой период. В первом случае все разделы заполнены, во втором выводится сообщение об отсутствии данных. Файлы открываются в Excel и просмотрщике PDF без ошибок.

3. Полученный результат

Статистика включена в Excel и PDF, значения совпадают с экранными. Отчёты корректно формируются для пустого периода.

4. Вывод

Экспорт статистики делает результаты анализа пригодными для хранения и передачи руководителю практики.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 32. Подключение журналирования Serilog

1. Цель и задачи этапа

Цель этапа — Настроить запись событий приложения в файл и консоль, определить уровни журналирования.

Для достижения цели решались следующие задачи:

1. подключить пакеты Serilog;
2. настроить вывод в файл с суточной ротацией;
3. записывать запуск, ошибки и ключевые действия;
4. проверить содержимое журнала.

2. Ход выполнения работ

Для журналирования использована библиотека Serilog с выводом в консоль и в файл. Файлы журнала располагаются в каталоге logs рядом с приложением и создаются отдельно для каждого дня. Срок хранения установлен 14 дней, после чего старые файлы удаляются автоматически.

Минимальный уровень записи — Information. В журнал попадают запуск приложения, путь к базе данных, количество пользователей после заполнения, ошибки интерфейса и необработанные исключения. Для необработанных ошибок используется уровень Fatal, для ошибок в работе формы — Error.

Шаблон записи включает дату и время, уровень, сообщение и текст исключения. Такая структура позволяет быстро найти причину сбоя: достаточно открыть файл за нужную дату и найти запись с уровнем ERR или FTL.

Проверка выполнялась запуском приложения, входом под разными пользователями и намеренным вводом неверного пароля. В журнале зафиксированы запуск, открытие формы и сообщение об ошибке входа без указания введённого пароля.

Листинг 1 — Настройка Serilog

```
Log.Logger = new LoggerConfiguration()
    .MinimumLevel.Information()
    .WriteTo.Console()
    .WriteTo.File(
        Path.Combine(AppPaths.LogDirectory, "app-.log"),
        rollingInterval: RollingInterval.Day,
        retainedFileCountLimit: 14)
    .CreateLogger();
```

3. Полученный результат

Журналирование настроено, файлы создаются и ротируются, ключевые события и ошибки фиксируются.

4. Вывод

Журнал упрощает диагностику ошибок и позволяет восстановить последовательность действий пользователя.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 33. Конфигурация приложения

1. Цель и задачи этапа

Цель этапа — Вынести настройки подключения к базе данных в файл конфигурации и организовать их чтение.

Для достижения цели решались следующие задачи:

1. создать файл appsettings.json;
2. подключить Microsoft.Extensions.Configuration;
3. обеспечить копирование файла при сборке;
4. предусмотреть значение по умолчанию.

2. Ход выполнения работ

Настройки приложения хранятся в файле appsettings.json. В текущей версии используется один параметр Database:Path, задающий путь к файлу базы данных относительно каталога приложения. Файл копируется в выходной каталог при сборке, поэтому его можно изменить без пересборки программы.

Чтение настроек выполнено через ConfigurationBuilder с методами SetBasePath и AddJsonFile. Если файл отсутствует, приложение использует значение по умолчанию data/repair-requests.db. Относительный путь преобразуется в абсолютный с учётом каталога приложения.

Секреты в файле не хранятся: база данных локальная и не требует пароля, а пароли пользователей сохраняются только в виде хеша. При необходимости подключения к серверной базе строку подключения следовало бы вынести в переменные среды, однако для учебного проекта это избыточно.

Проверено изменение пути к базе данных: после указания нового файла приложение создаёт базу в заданном каталоге и применяет миграцию. При отсутствии каталога он создаётся автоматически.

Листинг 1 — Файл appsettings.json

```
{
  "Database": {
    "Path": "data/repair-requests.db"
  }
}
```

3. Полученный результат

Конфигурация вынесена в отдельный файл, путь к базе данных читается при запуске, предусмотрено значение по умолчанию.

4. Вывод

Вынесение настроек упрощает перенос приложения и изменение параметров без изменения исходного кода.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 34. Интеграция с внешним API курсов валют

1. Цель и задачи этапа

Цель этапа — Реализовать получение курсов валют по HTTP с обработкой ошибок и таймаутом.

Для достижения цели решались следующие задачи:

1. изучить формат ответа выбранного API;
2. реализовать HTTP-запрос с таймаутом;
3. разобрать JSON и обработать ошибки;
4. предусмотреть резервные значения.

2. Ход выполнения работ

Для демонстрации внешней интеграции реализован сервис CurrencyService, который получает курсы доллара и евро с открытого источника cbr-xml-daily.ru. Ответ содержит дату котировок и значения валют в формате JSON. Сервис извлекает курс доллара и евро, проверяет, что значения положительны, и возвращает объект с датой и признаком источника.

HTTP-клиент создаётся один раз и переиспользуется. Таймаут установлен равным шести секундам, чтобы при отсутствии сети приложение не зависало. Запрос выполняется асинхронно, интерфейс в это время остаётся отзывчивым.

Обрабатываются три вида ошибок: сетевые исключения, превышение таймаута и некорректный JSON. Во всех случаях возвращается резервный курс 92,50 рубля за доллар и 100,20 рубля за евро с признаком резервного значения и текстом ошибки. Приложение не прерывает работу из-за недоступности внешнего сервиса.

Сервис зарегистрирован в контейнере приложения и покрыт модульными тестами на разбор корректного ответа и на обработку ошибок. В текущей версии интерфейса стоимость работ и запчастей вводится в рублях,

поэтому сервис используется как подготовленная интеграция и не влияет на основные сценарии работы.

Листинг 1 — Получение курса с резервным значением

```
try
{
    var json = await Http.GetStringAsync(Endpoint, cancellationToken);
    // разбор ответа и проверка значений
}
catch (Exception ex) when (ex is HttpRequestException
    or TaskCanceledException or JsonException)
{
    return Fallback with { Error = ex.Message };
}
```

3. Полученный результат

Сервис курсов реализован, таймаут и резервные значения настроены, ошибки не приводят к сбою приложения.

4. Вывод

Внешняя интеграция выполнена по принятой схеме: отдельный сервис, асинхронный вызов, ограничение времени и понятное поведение при сбое.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 35. Модульные тесты авторизации и валидации

1. Цель и задачи этапа

Цель этапа — Разработать модульные тесты для проверки авторизации, хеширования паролей и валидации данных.

Для достижения цели решались следующие задачи:

1. подготовить тестовый проект xUnit;
2. покрыть сценарии входа и регистрации;
3. проверить хеширование и проверку пароля;
4. проверить граничные значения валидаторов.

2. Ход выполнения работ

Тесты размещены в проекте RepairRequests.Tests и запускаются командой dotnet test. Для авторизации проверены успешный вход, неверный пароль, несуществующая почта, пустые поля и заблокированная учётная запись. Отдельно проверена регистрация: короткий пароль, отсутствие цифр или букв, несовпадение паролей и повторная регистрация на ту же почту.

Тесты хеширования проверяют, что одинаковые пароли дают разные хеши благодаря соли, что корректный пароль подтверждается, а неверный — отклоняется. Отдельно проверены пустая строка, строка неверного формата и повреждённое значение хеша. Всего выполнено пятнадцать таких проверок.

Тесты валидатора данных охватывают государственный номер, VIN, год выпуска, пробег, стоимость и длину текстовых полей. Для каждого правила подготовлены корректные и некорректные значения, включая граничные. Всего выполнено двадцать пять проверок, все они проходят.

Тесты не обращаются к рабочей базе данных и не зависят от порядка запуска. Каждый тест создаёт собственные объекты, поэтому результаты воспроизводимы. Общий запуск набора занимает около шестнадцати секунд.

Таблица 1 — Состав модульных тестов

Группа	Количество проверок

Группа	Количество проверок
Авторизация	17
Хеширование паролей	15
Валидация данных	25
Работа со статусами	20

3. Полученный результат

Модульные тесты авторизации, хеширования и валидации разработаны и проходят. Ошибки в основных проверках данных исключены.

4. Вывод

Модульные тесты позволяют изменять код без риска нарушить проверку учётных данных и валидацию форм.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 36. Интеграционные тесты взаимодействия заявки и справочников

1. Цель и задачи этапа

Цель этапа — Проверить взаимодействие заявок со справочниками через реальную базу данных.

Для достижения цели решались следующие задачи:

1. создать тестовую базу SQLite;
2. проверить сохранение и чтение связей;
3. проверить ограничения удаления;
4. проверить фильтрацию и сортировку.

2. Ход выполнения работ

Интеграционные тесты используют SQLite с отдельным файлом или соединением в памяти. Перед каждым тестом создаётся новая база, применяется модель и добавляются минимальные наборы пользователей, автомобилей и неисправностей. После завершения теста база удаляется.

Проверено сохранение заявки со ссылками на автомобиль, неисправность и клиента, а также чтение заявки с подгрузкой связанных данных. Отдельно проверено, что при удалении автомобиля, на который ссылается заявка, возникает ошибка ограничения, а при удалении заявки её история удаляется каскадно.

Тесты выборки проверяют фильтр по статусу, по диапазону дат, по клиенту и сортировку. Для каждого случая подготовлен набор из нескольких заявок с разными значениями, после чего сравнивается количество и порядок полученных записей.

Всего выполнено двадцать три интеграционные проверки. Они занимают больше времени, чем модульные, поскольку создают базу и выполняют реальные запросы, однако именно они подтверждают, что настройки связей и запросы работают так же, как в рабочей базе.

3. Полученный результат

Интеграционные тесты подтверждают корректность связей, ограничений и выборок. Ошибки настройки модели данных выявляются до запуска приложения.

4. Вывод

Сочетание модульных и интеграционных тестов обеспечивает проверку как отдельных функций, так и взаимодействия компонентов.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 37. Разработка тест-плана

1. Цель и задачи этапа

Цель этапа — Составить тест-план, охватывающий роли, фильтрацию, операции со данными и обработку ошибок.

Для достижения цели решались следующие задачи:

1. определить виды и объекты тестирования;
2. описать тестовые сценарии и ожидаемые результаты;
3. назначить тестовые данные;
4. оформить результаты проверки.

2. Ход выполнения работ

Тест-план включает функциональное тестирование, проверку прав доступа, тестирование форм и отчётов, а также проверку обработки ошибочных ситуаций. Для каждого сценария указаны предусловие, шаги, ожидаемый и фактический результат, отметка о прохождении.

Функциональные сценарии охватывают вход, регистрацию, добавление автомобиля, создание заявки, назначение механика, смену статуса, закрытие заявки, формирование отчёта и экспорт файлов. Отдельно проверены поиск, фильтрация по статусу и периоду, сортировка и ограничение видимости для клиента.

Проверка ошибок включает пустые поля, неверный формат номера и VIN, отрицательные стоимости, попытку удалить связанную запись, неверный пароль и недоступность внешнего сервиса курсов. Для каждой ситуации ожидается понятное сообщение и отсутствие аварийного завершения.

Все сценарии выполнены на тестовой базе из сорока двух заявок. Результаты зафиксированы в таблице тест-плана; сценарии, завершившиеся ошибкой, исправлены и проверены повторно. Итоговый тест-план приведён в приложении Г отчёта.

Таблица 1 — Основные проверки тест-плана

Проверка	Ожидаемый результат
Вход с верными данными	Открывается главное меню
Вход с неверным паролем	Сообщение об ошибке, вход не выполняется
Клиент открывает чужие заявки	Данные недоступны
Добавление автомобиля	Запись появляется в справочнике
Заявка без описания	Сохранение блокируется
Удаление связанного автомобиля	Удаление запрещено
Отчёт за пустой период	Нулевые показатели и сообщение
Экспорт в Excel и PDF	Файлы формируются и открываются

3. Полученный результат

Тест-план составлен и выполнен. Все основные сценарии проходят, обнаруженные замечания устранены.

4. Вывод

Тест-план позволил проверить систему по единому перечню и подтвердить готовность к защите.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 38. Отчёт по заявкам за период

1. Цель и задачи этапа

Цель этапа — Разработать форму отчёта с фильтром по периоду, сводными показателями и таблицами распределения.

Для достижения цели решались следующие задачи:

1. разместить выбор периода и кнопку формирования;
2. вывести сводные показатели;
3. построить таблицы по категориям, автомобилям и месяцам;
4. предусмотреть экспорт файлов.

2. Ход выполнения работ

Форма отчёта открывается из главного меню. В верхней части расположены элементы выбора начала и окончания периода, кнопки «Сформировать», «Excel» и «PDF». Период по умолчанию охватывает шесть месяцев, включая текущий.

После нажатия кнопки выполняется запрос к репозиторию с учётом периода и прав пользователя. Показатели выводятся в виде блоков: всего заявок, выполнено, активных, среднее время ремонта, медиана, отклонение, суммарная стоимость и средний простой.

Ниже расположены вкладки «По категориям», «По автомобилям» и «По месяцам». В таблицах приведены количество заявок, средний простой, среднее время ремонта, среднее ожидание и максимальный простой. Для каждой строки можно сравнить загруженность категории или автомобиля.

В нижней части формы отображается состояние: период, количество заявок и время формирования. При ошибке выводится сообщение, а предыдущие данные не уничтожаются. Отчёт для клиента строится только по его заявкам.

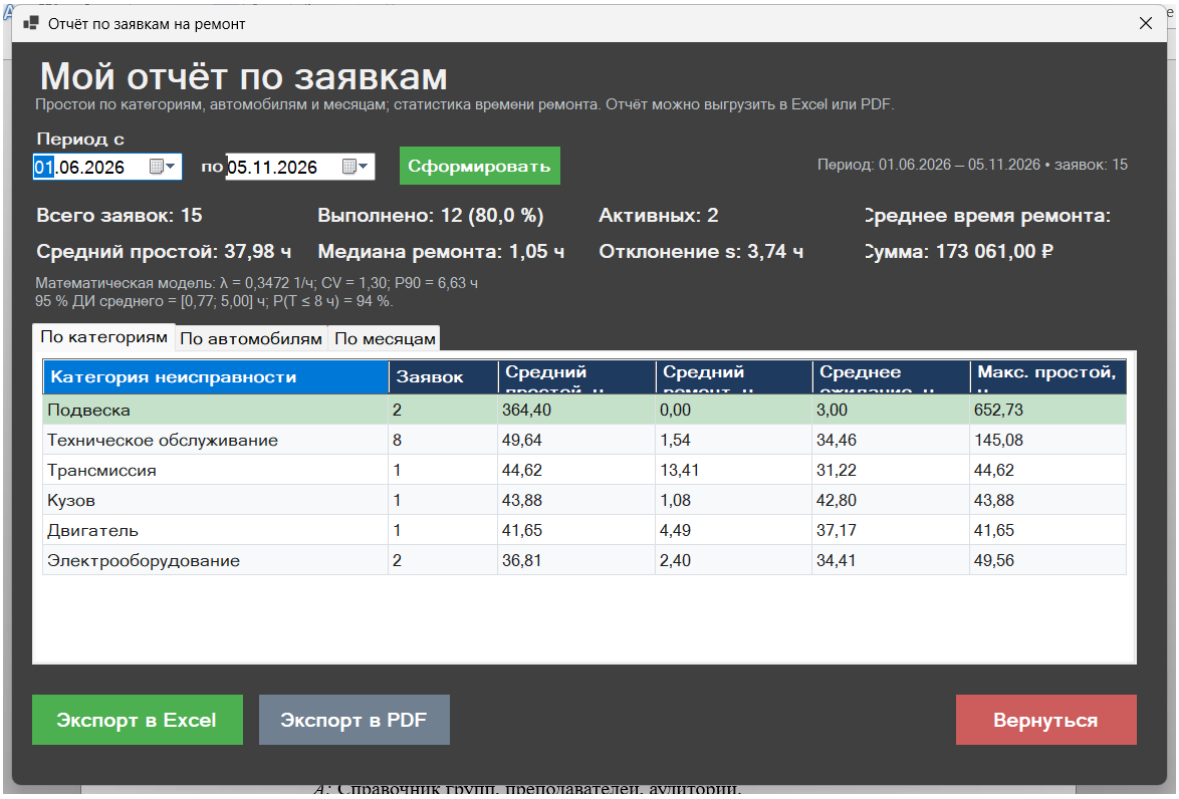


Рисунок 1 — Отчёт по заявкам клиента

3. Полученный результат

Форма отчёта работает, показатели и таблицы соответствуют выбранному периоду, экспорт доступен из той же формы.

4. Вывод

Отчёт объединяет оперативные данные и статистику, что позволяет оценивать загрузку станции и сроки ремонта.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 39. Экспорт отчёта в Excel

1. Цель и задачи этапа

Цель этапа — Реализовать выгрузку отчёта в книгу Excel с несколькими листами и оформлением.

Для достижения цели решались следующие задачи:

1. подключить библиотеку ClosedXML;
2. сформировать листы сводки, заявок, простоев и статистики;
3. настроить ширину столбцов и форматы;
4. проверить открытие файла.

2. Ход выполнения работ

Для выгрузки в Excel использована библиотека ClosedXML. Книга создаётся программно, без установленного пакета Microsoft Office. Имя файла содержит дату и время формирования, что исключает случайную перезапись предыдущего отчёта.

Книга состоит из четырёх листов. Лист «Сводка» содержит период, дату формирования, пользователя и основные показатели. Лист «Заявки» содержит построчный перечень заявок с номером, датами, автомобилем, клиентом, неисправностью, статусом и стоимостью.

Лист «Простои» содержит распределение по категориям неисправностей и автомобилям: количество заявок, средний простой, среднее время ремонта и ожидания. Лист «Статистика» содержит показатели описательной статистики и параметры экспоненциальной модели.

Для числовых столбцов заданы форматы с двумя знаками после запятой, для дат — формат dd.MM.yyyy. Ширина столбцов подобрана по содержимому, заголовки выделены границами. Проверено открытие файла в Excel: книга открывается без предупреждений, все листы доступны, формулы не требуются.

Таблица 1 — Состав книги Excel

Лист	Содержание
Сводка	Период, пользователь, итоговые показатели
Заявки	Построчный перечень заявок
Простои	Распределение по категориям и автомобилям
Статистика	Показатели и параметры модели

3. Полученный результат

Выгрузка в Excel работает, книга содержит четыре листа, форматы и ширины столбцов настроены.

4. Вывод

Экспорт в Excel позволяет хранить отчёты и обрабатывать данные средствами электронных таблиц.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 40. Экспорт отчёта в PDF

1. Цель и задачи этапа

Цель этапа — Реализовать формирование PDF-отчёта с разделами, таблицами и нумерацией страниц.

Для достижения цели решались следующие задачи:

1. подключить библиотеку QuestPDF;
2. сформировать титульную часть и разделы;
3. оформить таблицы и подписи;
4. проверить разбиение на страницы.

2. Ход выполнения работ

PDF-отчёт формируется библиотекой QuestPDF. Документ имеет формат A4, поля 20 мм, шрифт Times New Roman. В верхнем колонтитуле указывается название системы, в нижнем — номер страницы и дата формирования.

Отчёт включает четыре раздела: сводные показатели, распределение заявок, математическая модель времени ремонта и перечень заявок. Таблицы оформлены тонкими линиями, заголовки повторяются при переносе на следующую страницу.

В разделе модели приведены формулы и вероятности завершения ремонта за два, четыре и восемь часов. Значения выделены, рядом даны пояснения. Для длинных описаний предусмотрен перенос текста внутри ячеек.

Проверено формирование файла за период с данными и за пустой период, а также открытие документа в просмотрщике. Размер файла составляет около 140 КБ, количество страниц зависит от числа заявок в периоде.

3. Полученный результат

PDF-отчёт формируется, содержит разделы, таблицы и нумерацию страниц, корректно открывается.

4. Вывод

Экспорт в PDF позволяет получить готовый к печати документ без установки дополнительного программного обеспечения.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 41. Построение графиков по данным базы

1. Цель и задачи этапа

Цель этапа — Построить диаграммы, отражающие динамику заявок, распределение статусов и простои по категориям.

Для достижения цели решались следующие задачи:

1. выгрузить данные из базы;
2. подготовить четыре диаграммы;
3. оформить графики в чёрно-белой гамме;
4. проверить соответствие данных отчёту.

2. Ход выполнения работ

Для наглядности в отчёт и презентацию включены четыре диаграммы: количество созданных и завершённых заявок по месяцам, распределение заявок по статусам, средний простой по категориям неисправностей и гистограмма времени ремонта.

Исходные данные выгружены из рабочей базы SQLite. По ним построены столбчатые диаграммы и гистограмма: подписаны оси и значения, изображения сохранены в формате PNG.

Диаграммы оформлены в чёрно-белой гамме: столбцы залиты оттенками серого, контуры чёрные, подписи выполнены шрифтом Times New Roman. Такой вариант соответствует требованию к оформлению учебных документов и не отвлекает внимание от данных.

Значения на диаграммах сверены с отчётом: за шесть месяцев создано 42 заявки, завершено 34, наибольший средний простой относится к категории «Кузов», среднее время ремонта составляет 2,81 часа. Расхождений с таблицами отчёта нет.

Таблица 1 — Состав диаграмм

Диаграмма	Показатель
Заявки по месяцам	Создано и завершено за месяц

Диаграмма	Показатель
Статусы заявок	Количество заявок по статусам
Простои по категориям	Средний простой, часы
Гистограмма времени	Распределение длительности ремонта

3. Полученный результат

Четыре диаграммы построены по реальным данным и оформлены в едином стиле. Значения соответствуют отчётной форме.

4. Вывод

Графики упрощают восприятие статистики и используются в отчёте и презентации при защите практики.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 42. Анализ статистики и оформление графиков в отчёте

1. Цель и задачи этапа

Цель этапа — Проанализировать построенные диаграммы и включить их в отчёт с краткими выводами.

Для достижения цели решались следующие задачи:

1. описать динамику заявок;
2. сравнить простои по категориям;
3. оценить разброс времени ремонта;
4. сформулировать выводы.

2. Ход выполнения работ

Диаграмма по месяцам показывает, что наибольшее число заявок создано в октябре 2026 года — девять, а в ноябре зафиксировано три заявки, поскольку месяц ещё не завершён. Количество завершённых заявок немного отстаёт от созданных, что объясняется нахождением части заявок в работе.

Распределение по статусам показывает, что большинство заявок закрыто — тридцать из сорока двух. В работе находятся две заявки, новые — две, отменённые — три. Такое распределение типично для базы, в которой накоплена история обслуживания за несколько месяцев.

Наибольший средний простой относится к категории «Кузов» — около 42,5 часа, наименьший — к тормозной системе, около 7,8 часа. Кузовные работы требуют ожидания деталей и согласования со страховыми компаниями, поэтому их длительность существенно выше слесарных работ.

Гистограмма времени ремонта имеет выраженную правостороннюю асимметрию: большинство ремонтов завершается за один-два часа, но отдельные заявки выполняются значительно дольше. Поэтому среднее значение 2,81 часа выше медианы 1,69 часа, а коэффициент вариации близок к единице.

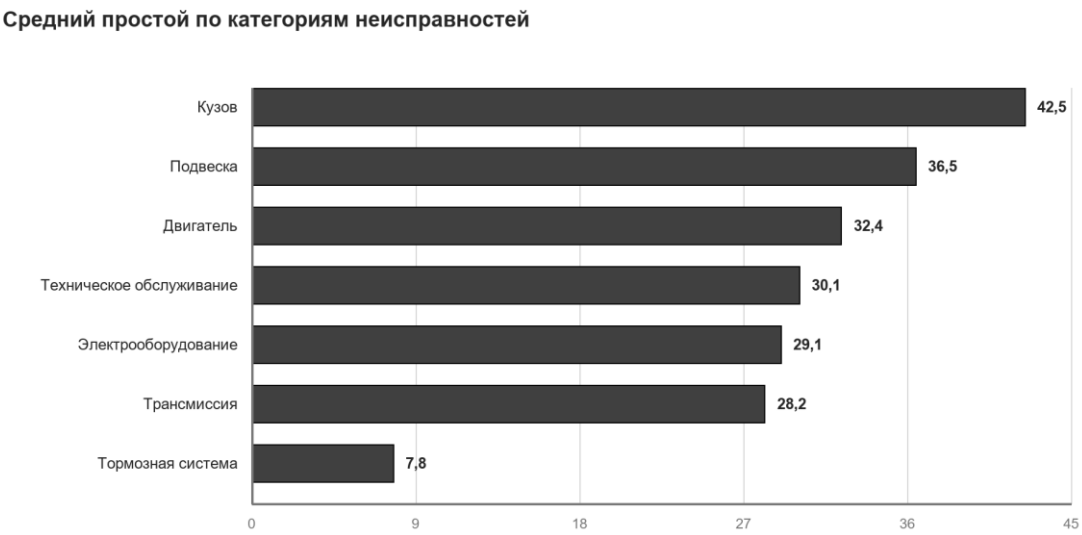


Рисунок 1 — Средний простой по категориям неисправностей

3. Полученный результат

Диаграммы проанализированы, выявлены наиболее длительные категории работ и характер распределения времени ремонта.

4. Вывод

Анализ показал, что для оценки сроков следует использовать медиану и процентили, а среднее значение дополнять показателями разброса.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 43. Руководство пользователя

1. Цель и задачи этапа

Цель этапа — Подготовить руководство, описывающее работу с системой для клиента и сотрудника станции.

Для достижения цели решались следующие задачи:

1. описать вход и регистрацию;
2. описать основные операции;
3. рассмотреть типичные ошибки;
4. дать рекомендации по работе.

2. Ход выполнения работ

Руководство начинается с описания назначения системы и требований к рабочему месту: Windows 10 или 11, разрешение экрана не ниже 1366×768, свободное место для базы данных и журналов. Затем приведены демонстрационные учётные записи для клиента, менеджера и администратора.

Раздел входа описывает ввод почты и пароля, переход к регистрации и возможные сообщения. Раздел клиента включает добавление автомобиля, оформление заявки, просмотр статуса и формирование отчёта по своим заявкам. Раздел сотрудника — обработку заявок, назначение механика, смену статусов и отчёты за период.

Отдельный раздел посвящён экспорту в Excel и PDF, выбору периода и проверке полученных файлов. Рассмотрены аварийные ситуации: отсутствие прав на файл базы данных, повреждение файла, недоступность внешнего сервиса курсов, некорректный ввод. Для каждой ситуации указаны причина и порядок действий.

В конце приведены рекомендации: регулярно копировать файл базы данных, не изменять файлы в каталоге данных вручную, закрывать приложение перед резервным копированием, использовать сложные пароли.

Руководство оформлено отдельным документом и включено в комплект отчётной документации.

3. Полученный результат

Руководство пользователя подготовлено, содержит описание операций, аварийных ситуаций и рекомендации.

4. Вывод

Руководство позволяет новому пользователю освоить основные операции без дополнительного обучения.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 44. Описание программы по ГОСТ 19.402-78

1. Цель и задачи этапа

Цель этапа — Оформить описание программы в соответствии со структурой ГОСТ 19.402-78.

Для достижения цели решались следующие задачи:

1. описать общие сведения и назначение;
2. описать логическую структуру;
3. указать технические средства и вызов;
4. описать входные и выходные данные.

2. Ход выполнения работ

Документ включает разделы, предусмотренные ГОСТ 19.402-78: общие сведения, функциональное назначение, описание логической структуры, используемые технические средства, вызов и загрузка, входные данные, выходные данные.

В общих сведениях указаны наименование программы «Система заявок на ремонт», язык программирования C#, среда .NET 10, библиотека Windows Forms и база данных SQLite. Функциональное назначение — учёт заявок на ремонт транспорта, ведение справочников, контроль статусов и формирование отчётности.

Описание логической структуры включает слои приложения: модели данных, репозитории, сервисы и формы. Показано, что интерфейс не содержит запросов к базе напрямую, а использует репозитории через контейнер служб. Для входных данных перечислены учётные данные, сведения об автомобиле и заявке; для выходных — отчёты на экране, файлы Excel и PDF, записи журнала.

Раздел «Вызов и загрузка» описывает запуск исполняемого файла, автоматическое применение миграции и заполнение базы при первом

запуске. Документ оформлен отдельным файлом и входит в комплект отчётной документации.

3. Полученный результат

Описание программы оформлено по структуре ГОСТ 19.402-78 и содержит все обязательные разделы.

4. Вывод

Документ фиксирует назначение, устройство и условия применения программы в принятой нормативной форме.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 45. Оформление листингов ключевых фрагментов кода

1. Цель и задачи этапа

Цель этапа — Подготовить листинги основных фрагментов кода с комментариями для пояснительной записки.

Для достижения цели решались следующие задачи:

1. отобрать ключевые фрагменты;

2. оформить листинги с заголовками;

3. добавить комментарии;

4. проверить корректность приведённого кода.

2. Ход выполнения работ

Для приложения подготовлены листинги, показывающие наиболее важные решения: хеширование и проверку пароля, расчёт статистики времени ремонта, построение запроса с фильтрацией, смену статуса заявки и формирование отчёта.

Каждый листинг имеет заголовок с номером и кратким названием, оформлен моноширинным шрифтом с одинарным интервалом. Фрагменты сокращены до существенных строк, служебный код и обработка интерфейсных событий опущены. Комментарии поясняют назначение формул и ограничений.

Код в листингах соответствует текущей версии проекта. Проверка выполнялась сверкой с исходными файлами и повторной сборкой решения: приведённые фрагменты компилируются в составе проекта без изменений.

Листинги включены в пояснительную записку и приложения. Для длинных фрагментов применён перенос строк, чтобы текст не выходил за поля страницы.

Листинг 1 — Смена статуса заявки

```
public static void Apply(RepairRequest request, RequestStatus target,
    int changedById, string? comment = null)
{
    if (!request.Status.CanTransitionTo(target))
```

```
            throw new InvalidOperationException(
                $"Переход {request.Status} -> {target} недопустим.");

        request.Status = target;
        request.History.Add(new RequestStatusHistory
        {
            Status = target,
            ChangedAt = DateTime.Now,
            ChangedById = changedById,
            Comment = comment
        });
    }
```

3. Полученный результат

Листинги подготовлены и оформлены, приведённый код соответствует исходникам проекта.

4. Вывод

Листинги позволяют проверить реализацию ключевых алгоритмов без просмотра всего исходного кода.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 46. Сборка релиза и публикация приложения

1. Цель и задачи этапа

Цель этапа — Настроить конфигурацию Release и подготовить папку публикации для переноса приложения.

Для достижения цели решались следующие задачи:

1. проверить сборку в конфигурации Release;
2. выполнить публикацию для Windows x64;
3. проверить состав файлов;
4. запустить опубликованное приложение.

2. Ход выполнения работ

Сборка выполнена командой dotnet publish с конфигурацией Release, целевой платформой win-x64 и режимом развёртывания, зависящим от установленной среды .NET. Такой вариант уменьшает размер папки публикации, но требует наличия рабочего стола .NET 10 на компьютере пользователя.

Команда публикации: dotnet publish src/RepairRequests.WinForms -c Release -r win-x64 --self-contained false -o publish/win-x64. В результате получено 55 файлов общим размером около 34 МБ. В папке находятся исполняемый файл, библиотеки приложения и сторонних пакетов, файл appsettings.json и файлы ресурсов.

После публикации проверено, что в папку не попали исходные файлы, тесты и лишние копии базы данных. База данных создаётся при первом запуске в каталоге data, журналы — в каталоге logs. Эти каталоги не входят в поставку и создаются автоматически.

Опубликованное приложение запущено на чистом каталоге: база создана, миграция применена, тестовые данные загружены, вход выполнен под демонстрационной учётной записью. Отчёты и экспорт работают так же, как в отладочной версии.

Таблица 1 — Параметры публикации

Параметр	Значение
Конфигурация	Release
Платформа	win-x64
Режим	Зависит от среды .NET
Папка	publish/win-x64
Файлов	55
Размер	около 34 МБ

3. Полученный результат

Релиз собран и опубликован, состав файлов проверен, приложение запускается из папки публикации.

4. Вывод

Публикация позволяет передать приложение на другой компьютер без среды разработки.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 47. Инструкция по развёртыванию

1. Цель и задачи этапа

Цель этапа — Подготовить инструкцию по установке и запуску приложения на рабочем месте.

Для достижения цели решались следующие задачи:

1. описать требования к компьютеру;
2. описать порядок установки;
3. описать настройку базы данных;
4. предусмотреть резервное копирование и обновление.

2. Ход выполнения работ

Для работы приложения требуется компьютер под управлением Windows 10 или 11, процессор с частотой не ниже 1,5 ГГц, не менее 4 ГБ оперативной памяти, 200 МБ свободного места и установленная среда .NET Desktop Runtime 10. Разрешение экрана рекомендуется не ниже 1366×768.

Установка заключается в копировании папки публикации на локальный диск. Запуск выполняется файлом RepairRequests.WinForms.exe. При первом запуске создаются каталоги data и logs, применяется миграция и загружаются тестовые данные. Если каталог программы защищён от записи, приложение выводит сообщение о невозможности создать базу.

Путь к базе данных задаётся в файле appsettings.json. Для переноса данных достаточно скопировать файл repair-requests.db в каталог data на новом компьютере. Резервное копирование выполняется копированием файла базы при закрытом приложении; для повышения надёжности рекомендуется хранить копии за несколько дней.

Обновление версии выполняется заменой файлов приложения с сохранением каталога data. Перед заменой база данных копируется в отдельный каталог. Если новая версия содержит миграцию, она применяется

автоматически при запуске. Инструкция оформлена отдельным документом и включена в приложение Д отчёта.

Таблица 1 — Минимальные требования

Компонент	Требование
Операционная система	Windows 10 или 11, 64-бит
Среда выполнения	.NET Desktop Runtime 10
Оперативная память	4 ГБ
Свободное место	200 МБ
Разрешение экрана	1366×768

3. Полученный результат

Инструкция по развёртыванию подготовлена и проверена на чистом каталоге. Порядок резервного копирования и обновления описан.

4. Вывод

Инструкция позволяет установить приложение и сохранить данные при переносе или обновлении версии.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 48. Инструкция по технике безопасности

1. Цель и задачи этапа

Цель этапа — Описать правила безопасной работы с персональным компьютером и электробезопасности.

Для достижения цели решались следующие задачи:

1. изучить требования к рабочему месту;
2. описать правила работы за компьютером;
3. описать действия при аварийных ситуациях;
4. оформить инструкцию.

2. Ход выполнения работ

Рабочее место должно быть оборудовано столом с регулируемой высотой, стулом с поддержкой поясницы, монитором с регулировкой наклона и местным освещением. Расстояние от глаз до экрана рекомендуется 60–70 см, верхний край монитора — на уровне глаз или slightly ниже.

Перед началом работы проверяется целостность кабелей и вилок, отсутствие повреждений корпуса компьютера и розеток. Запрещается работать при снятой крышке корпуса, касаться разъёмов влажными руками, подключать неисправные устройства и оставлять работающий компьютер без присмотра.

При работе необходимо делать перерывы по 10–15 минут каждый час, выполнять упражнения для глаз и кистей рук. Непрерывная работа за экраном не должна превышать двух часов. При появлении усталости или боли в глазах следует прекратить работу и отдохнуть.

При аварийной ситуации необходимо отключить питание, сообщить руководителю и действовать по инструкции по эвакуации. При возгорании нельзя тушить электрооборудование водой; используются углекислотные огнетушители. Инструкция оформлена отдельным документом.

3. Полученный результат

Инструкция по технике безопасности подготовлена и включает требования к рабочему месту, правила работы и действия при аварии.

4. Вывод

Соблюдение требований охраны труда снижает риск травм и профессиональных заболеваний при работе с компьютером.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 49. Проверка структуры проекта и исправление ошибок

1. Цель и задачи этапа

Цель этапа — Проверить структуру решения, целостность документации и устранить обнаруженные ошибки.

Для достижения цели решались следующие задачи:

1. сверить состав файлов с описанием;
2. проверить сборку без предупреждений;
3. проверить отчёты на пустом периоде;
4. исправить найденные ошибки.

2. Ход выполнения работ

Проверено соответствие структуры решения описанию: проекты Core, WinForms и Tests, каталоги моделей, данных, репозиториев, сервисов, форм и инфраструктуры. Лишние файлы, временные каталоги и копии базы данных в репозиторий не попадают.

Сборка выполнена с уровнем предупреждений, принятым в проекте. Предупреждения о неиспользуемых переменных и несоответствии типов устранены. Отдельно проверено, что формы не содержат прямых запросов к базе данных и работают через репозитории.

При проверке отчёта на пустом периоде обнаружено деление на количество наблюдений при отсутствии завершённых заявок. Расчёт статистики изменён: при пустой выборке возвращается объект с нулевыми значениями, а форма выводит сообщение о недостатке данных.

Также исправлено сообщение при попытке удалить автомобиль, связанный с заявками: вместо технического текста ошибки базы данных выводится понятное пояснение. После исправлений повторно выполнены сборка и полный набор тестов.

3. Полученный результат

Структура проекта проверена, найденные ошибки исправлены, сборка и тесты проходят без ошибок.

4. Вывод

Проверка структуры и граничных случаев повысила устойчивость приложения перед защитой.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 50. Рефакторинг кода

1. Цель и задачи этапа

Цель этапа — Устранить дублирование кода и улучшить читаемость основных классов.

Для достижения цели решались следующие задачи:

1. выделить повторяющиеся элементы интерфейса;
2. упростить работу с базой данных;
3. привести имена к единому стилю;
4. повторно выполнить тесты.

2. Ход выполнения работ

В формах повторялись настройки шрифтов, цветов, размеров кнопок и таблиц. Общие настройки вынесены в класс UiTheme, который создаёт кнопки, поля, подписи и таблицы в едином стиле. Формы стали короче, а внешний вид — единообразным.

Работа с базой данных упрощена за счёт фабрики контекста и репозиториев. Повторяющиеся запросы получения списка и поиска по идентификатору вынесены в базовые методы. В формах остались только вызовы репозиториев и обработка результатов.

Имена классов, методов и переменных приведены к стилю C#: классы и методы — PascalCase, параметры и локальные переменные — camelCase, приватные поля — с подчёркиванием. Комментарии оставлены для неочевидных решений: расчёт статистики, правила переходов статусов, обработка ошибок внешнего сервиса.

После рефакторинга выполнены сборка и все тесты. Количество тестов не уменьшилось, результаты прохождения не изменились. Повторно проверены вход, создание заявки, смена статуса и формирование отчётов.

3. Полученный результат

Дублирование сокращено, интерфейс приведён к единому стилю, имена соответствуют соглашениям C#. Тесты проходят.

4. Вывод

Рефакторинг упростил сопровождение проекта и снизил риск ошибок при дальнейших изменениях.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 51. Исправление замечаний и подготовка документации

1. Цель и задачи этапа

Цель этапа — Устранить замечания руководителя практики и привести документы в соответствие требованиям.

Для достижения цели решались следующие задачи:

1. собрать и систематизировать замечания;
2. исправить текст и оформление;
3. синхронизировать документы с кодом;
4. проверить итоговый комплект.

2. Ход выполнения работ

Замечания руководителя касались оформления таблиц, единообразия подписей к рисункам и полноты описания статистики. Замечания сведены в список, для каждого определено действие и проверен результат после исправления.

В документах убрано полужирное начертание в основном тексте и таблицах, оставлено только для заголовков разделов и подразделов. Диаграммы и графики приведены к чёрно-белой гамме, таблицы оформлены без заливки и лишних линий. Подписи рисунков и таблиц приведены к единому виду.

Тексты разделов соответствуют фактическому поведению программы: указаны реальные названия форм, полей и файлов, количество тестов и объём данных. Скриншоты отражают текущий интерфейс.

Проверен итоговый комплект: пояснительная записка, документы по этапам, руководство пользователя, описание по ГОСТ, тест-план, инструкция по развёртыванию и презентация. Все документы открываются, оглавление обновлено, нумерация страниц соответствует содержанию.

3. Полученный результат

Замечания устранены, документы приведены к единому стилю и соответствуют текущей версии программы.

4. Вывод

Исправление замечаний обеспечило согласованность кода, текста и иллюстраций в итоговом комплекте.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 52. Подготовка презентации к защите

1. Цель и задачи этапа

Цель этапа — Создать презентацию, отражающую тему, задачи, реализацию и результаты практики.

Для достижения цели решались следующие задачи:

1. определить структуру слайдов;
2. подготовить текст и иллюстрации;
3. оформить слайды в едином стиле;
4. проверить читаемость и объём.

2. Ход выполнения работ

Презентация рассчитана на выступление продолжительностью 7–8 минут и содержит 15 слайдов. Первый слайд — тема, фамилия студента, группа и руководитель. Далее следуют актуальность, цель и задачи, предметная область, структура системы и схема базы данных.

Отдельные слайды посвящены ролям пользователей, жизненному циклу заявки, расчёту статистики и формулам экспоненциальной модели. Реализация показана скриншотами главного меню, списка заявок, формы заявки и отчёта. Результаты представлены диаграммами и краткими выводами.

Для слайдов использованы чёрно-белые диаграммы и скриншоты интерфейса. Текст краткий, не более пяти строк на слайд, шрифт не меньше 20 пунктов. Единый стиль оформления: белый фон, тёмный текст, тонкие линии, один акцентный элемент.

Презентация проверена на проекторе и на экране ноутбука: текст читается, изображения не размыты, слайды не перегружены. Файл сохранён в формате PPTX и дополнительно экспортирован в PDF.

3. Полученный результат

Презентация из 15 слайдов подготовлена, оформлена в едином стиле и проверена на читаемость.

4. Вывод

Презентация позволяет за короткое время показать тему, реализацию и полученные результаты.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 53. Пробное выступление и подготовка ответов на вопросы

1. Цель и задачи этапа

Цель этапа — Проверить длительность выступления и подготовить ответы на вероятные вопросы.

Для достижения цели решались следующие задачи:

1. провести выступление с хронометражем;
2. сократить или дополнить текст;
3. подготовить ответы на вопросы;
4. проверить демонстрацию программы.

2. Ход выполнения работ

Пробное выступление проведено с хронометражем. Вступление и постановка задачи заняли около минуты, проектирование — две минуты, реализация — три минуты, результаты и выводы — полторы минуты. Общая длительность уложилась в восемь минут.

По результатам хронометража сокращён текст на слайдах о нормализации и тестировании, добавлено пояснение о выборе экспоненциальной модели. Отдельно подготовлен короткий вариант выступления на случай ограничения времени до пяти минут.

Подготовлены ответы на вопросы о выборе SQLite, защите паролей, ограничении прав клиента, причинах различия среднего и медианы, способах резервного копирования и переноса базы. Для каждого вопроса указаны слайд или раздел отчёта, где приведены подробности.

Демонстрация проверена в порядке, предусмотренном выступлением: вход под менеджером, список заявок, создание заявки, смена статуса, отчёт за период, экспорт в Excel и PDF. Проверено, что демонстрационная база содержит нужные записи и отчёт формируется без задержек.

3. Полученный результат

Выступление проверено по времени, текст скорректирован, ответы на вопросы подготовлены, демонстрация отрепетирована.

4. Вывод

Пробное выступление позволило устранить спешку и подготовить доказательные ответы по ключевым решениям проекта.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.

Этап 54. Защита отчёта и демонстрация работы системы

1. Цель и задачи этапа

Цель этапа — Представить результаты учебной практики и продемонстрировать работу разработанной системы.

Для достижения цели решались следующие задачи:

1. представить тему и задачи практики;
2. показать основные функции системы;
3. привести результаты статистики;
4. ответить на вопросы комиссии.

2. Ход выполнения работ

На защите представлены тема практики, цель и задачи, краткая характеристика предметной области и обоснование необходимости разработки. Показаны структура системы, схема базы данных и распределение ролей.

В ходе демонстрации выполнен вход под менеджером, открыт список заявок, создана новая заявка с выбором автомобиля и неисправности, назначен механик и изменён статус. Затем открыт отчёт за период, показаны сводные показатели, распределение по категориям и вероятности завершения ремонта.

Отдельно продемонстрированы экспорт отчёта в Excel и PDF, а также работа под учётной записью клиента: клиент видит только свои автомобили и заявки. Показано, что попытка удалить связанный автомобиль блокируется и сопровождается понятным сообщением.

В завершение приведены количественные результаты: 42 заявки в тестовой базе, 34 завершённых ремонта, среднее время ремонта 2,81 часа, медиана 1,69 часа, 133 автоматических теста. Отмечены направления развития: управление пользователями, складской учёт запчастей, уведомления клиентам.

3. Полученный результат

Отчёт защищён, работа системы продемонстрирована, результаты практики представлены комиссии.

4. Вывод

Защита подтвердила работоспособность системы и соответствие результатов поставленным задачам.

Обучающийся: _____ / _____

Дата: «____» _____ 2026 г.